

Vel Tech Multi Tech

Dr.Rangarajan Dr.Sakunthala Engineering College

An Autonomous Institution

(Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai)

ISO9001:2015 Certified Institution, Accredited by NBA (BME, CSE, ECE, EEE, IT&MECH),

Accredited by NAAC with 'A' Grade with CGPA of 3.49

#42, Avadi – Vel Tech Road, Avadi, Chennai-600062, Tamil Nadu, India.

DEPARTMENT OF INFORMATION TECHNOLOGY



231IT52A - NETWORKS LABORATORY

NAME :
REGISTER NO :
ROLL NO : VM -
BRANCH : B.Tech – INFORMATION TECHNOLOGY
YEAR :
SEMESTER :

DEPARTMENT OF INFORMATION TECHNOLOGY

VISION

- To emerge as Centre for academic eminence in the field of information technology through innovative learning practices.

MISSION

- **M1**-To provide good teaching and learning environment for quality education in the field of information technology.
- **M2**-To propagate lifelong learning.
- **M3**-To impart the right proportion of knowledge, attitudes and ethics in students to enable them take up positions of responsibility in the society and make significant contributions.

Vel Tech Multi Tech

Dr.Rangarajan Dr.Sakunthala Engineering College

An Autonomous Institution

(Approved by AICTE, New Delhi & Affiliated to Anna University, Chennai)

ISO9001:2015 Certified Institution, Accredited by NBA (BME, CSE, ECE, EEE, IT & MECH),

Accredited by NAAC with 'A' Grade with CGPA of 3.49

#42, Avadi - Vel Tech Road, Avadi, Chennai-600062, Tamil Nadu, India.

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

Name:

Year: Semester: , Branch: **B. Tech–INFORMATION TECHNOLOGY**

University Register No: College Roll No:

Certified that this is the bonafide record of work done by the above student in the

23IIT52A – NETWORKS LABORATORY during the academic year **2025-2026**.

Signature of Head of the Department

Signature of Staff In-charge

Submitted for the University Practical Examination held on at VEL TECH MULTI
TECH Dr.RANGARAJAN Dr .SAKUNTHALA ENGINEERING COLLEGE, #42, AVADI –
VEL TECH ROAD, AVADI, CHENNAI- 600062.

Signature of Examiners:

INTERNAL EXAMINER

EXTERNAL EXAMINER

Date:

DEPARTMENT OF INFORMATION TECHNOLOGY

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

PEOs	PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)
PEO 1	Graduates will demonstrate technical competency and leadership skills to lead a successful career in the field of IT& ITES.
PEO 2	Graduate will exhibit a commitment to communicate effectively in diverse environment and apply proficiency towards societal issue with human values.
PEO 3	Graduates will pursue lifelong learning in generating innovative solutions to the changing industrial needs using research and problem-solving skills.

PROGRAMME SPECIFIC OUTCOMES (PSOs)

PSO's	PROGRAMME SPECIFIC OUTCOMES
PSO 1	An ability to apply design and development principles in the construction of software systems of varying complexity.
PSO 2	The use of current application software, the design and use of operating systems and the analysis, design, testing and documentation of computer programs for the use in the information engineering technologies.
PSO 3	The design techniques, analysis and the building, testing, operation and maintenance of networks, databases, security and computer systems. (both hardware and software).

PROGRAMME OUTCOMES (POs)

PO's	PROGRAMME OUTCOMES(POs)
PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required.
PO4	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions.
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

231IT52A - NETWORKS LABORATORY

COURSE OBJECTIVES

The student should be made to:

- To learn network commands
- To learn socket programming
- To implement and analyze various network protocols
- To learn and use simulation tools
- To use simulation tools to analyze the performance of various network protocols

COURSE OUTCOMES

At the end of the course, the student should be able to

CO1 – Implement basic network commands and simulates error correction codes.

CO2 - Simulate data link layer protocols and routing algorithms.

CO3 – Analyse TCP and UDP performance and simulate Application layer protocol

Course Outcome	Mapping CO's with PO's and PSO's													
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2	PSO 3
CO1	3	2	1	-	2	-	-	-	-	-	-	3	-	-
CO2	3	2	2	1	2	-	-	-	2	2	2	3	2	1
CO3	3	3	2	1	2	-	-	-	2	2	2	3	2	2
CO	3	2	2	1	2	-	-	-	2	2	2	3	2	2

23IIT52A–NETWORKS LABORATORY

INDEX

Ex. No	Date	List of Exercises	Page No	Mark	Signature
1		Learn to use commands like tcpdump, netstat, ifconfig, nslookup and traceroute. Capture ping and trace route PDUs using a network protocol analyzer and examine.			
2		Write a HTTP web client program to download a web page using TCP sockets.			
3		Applications using TCP Sockets like: Echo client and Echo server Chat File Transfer			
4		Simulation of DNS using UDP sockets			
5		Use a tool like Wireshark to capture packets and examine the packets.			
6		Write a code simulating ARP/RARP protocols			
7		Study of Network Simulator (NS) and Simulation of Congestion Control Algorithms using NS			
8		Study of TCP/UDP performance using simulation tool			
9		Study of Distance vector/Link State routing algorithm using NS			
10		Simulation of Error Correction Code(CRC)			
11		Implementations of Stop and Wait Protocol and Sliding Window Protocol			
12		Write a code simulating Go Back N ARQ			
13		CASE STUDY / MINI PROJECT			

EX NO: 1

LEARN TO USE COMMANDS LIKE TCPDUMP, NETSTAT, IFCONFIG, NSLOOKUP AND TRACEROUTE. CAPTURE PING AND TRACE ROUTE PDUS USING A NETWORK PROTOCOL ANALYZER AND EXAMINE.

Aim:

To learn to use commands like tcpdump, netstat, ipconfig, nslookup and traceroute ping.

Algorithm - Commands:

1. Tcpdump

Tcpdump is a powerful, command-line packet analyzer used for capturing and analyzing network traffic, crucial for troubleshooting and understanding network activity.

#tcpdump icmp

❖ **Display traffic between two hosts:**

To display all traffic between two hosts (represented by variables host 1 and host 2)

#tcpdump host host1 and host 2

❖ **Display traffic between source and destination host only:**

To display the traffic between source(src) and destination (dst) host

#tcpdump src host

#tcpdump dst host

❖ **Display traffic for a specific protocol:**

Provide the protocol as an argument to display the traffic for a specific protocol,

For example: tcp, udp, icmp, arp

#tcpdump protocol

For example to display traffic only for the tcp traffic:

#tcpdump tcp

❖ **Filtered based on source or destination port:**

To filter based on a source or destination port:

#tcpdump src port ftp

#tcpdump dst port http

2. Netstat:

Netstat is a common command line TCP/IP networking available in versions of Windows, Linux, Unix and other operating systems.

#netstat

3. Ipconfig:

From the command prompt, type ipconfig to run the utility with default options. The output of the default command contains the IP address, network mask and gateway for all physical and network adapter.

#ipconfig

4. nslookup:

The nslookup(stands for name server lookup) command is a network utility program used to obtain information about internet servers. It finds name server information for domains by querying the Domain Name System.

#nslookup

5. Traceroute:

The TRACERT(stands for trace route) command is a command line utility to trace the path of an Internet Protocol(IP) packet as it traverses routers locally or globally. It is primarily used for diagnostic and troubleshooting purposes.

#tracert google.com

6. Ping:

The Ping is a command-line utility, available on virtually any operating system with network connectivity, that acts as a test to see if a networked device is reachable. The ping command sends a request over the network to a specific device. A successful ping results in a response from the computer that was pinged back to the originating computer.

ping 172.16.6.2

OUTPUT - Commands:

1. NETSTAT:

```
Command Prompt

C:\Users\Students>netstat

Active Connections

Proto Local Address           Foreign Address         State
TCP    127.0.0.1:49698          kubernetes:49699       ESTABLISHED
TCP    127.0.0.1:49699          kubernetes:49698       ESTABLISHED
TCP    127.0.0.1:49711          kubernetes:49712       ESTABLISHED
TCP    127.0.0.1:49712          kubernetes:49711       ESTABLISHED
TCP    172.16.108.201:7680      DESKTOP-58B4SPV:50184  ESTABLISHED
TCP    172.16.108.201:7680      172.16.112.92:50202    TIME_WAIT
TCP    172.16.108.201:49716     20.90.156.32:https      ESTABLISHED
TCP    172.16.108.201:49810     ec2-54-169-5-36:http    ESTABLISHED
TCP    172.16.108.201:49833     ec2-52-35-58-119:https  CLOSE_WAIT
TCP    172.16.108.201:49834     104.18.63.125:https     CLOSE_WAIT
TCP    172.16.108.201:49836     ecs-90-84-189-232:https CLOSE_WAIT
TCP    172.16.108.201:49840     104.18.63.125:https     CLOSE_WAIT
TCP    172.16.108.201:50063     wf-in-f188:5228         ESTABLISHED
TCP    172.16.108.201:50078     172.16.106.111:8009     ESTABLISHED
TCP    172.16.108.201:50518     ec2-54-77-52-141:http   ESTABLISHED
TCP    172.16.108.201:50799     104.18.63.131:https     CLOSE_WAIT
TCP    172.16.108.201:50897     151.101.193.229:https   ESTABLISHED
TCP    172.16.108.201:50906     226:https               ESTABLISHED
TCP    172.16.108.201:50935     ecs-90-84-175-86:https  CLOSE_WAIT
TCP    172.16.108.201:50936     server-3-164-163-38:https CLOSE_WAIT
TCP    172.16.108.201:50937     server-3-164-163-38:https CLOSE_WAIT
TCP    172.16.108.201:50938     server-3-164-163-38:https CLOSE_WAIT
TCP    172.16.108.201:50939     server-3-164-163-38:https CLOSE_WAIT
TCP    172.16.108.201:50940     server-3-164-163-38:https CLOSE_WAIT
```

2. IPCONFIG:

```
Command Prompt

C:\Users\Students>ipconfig

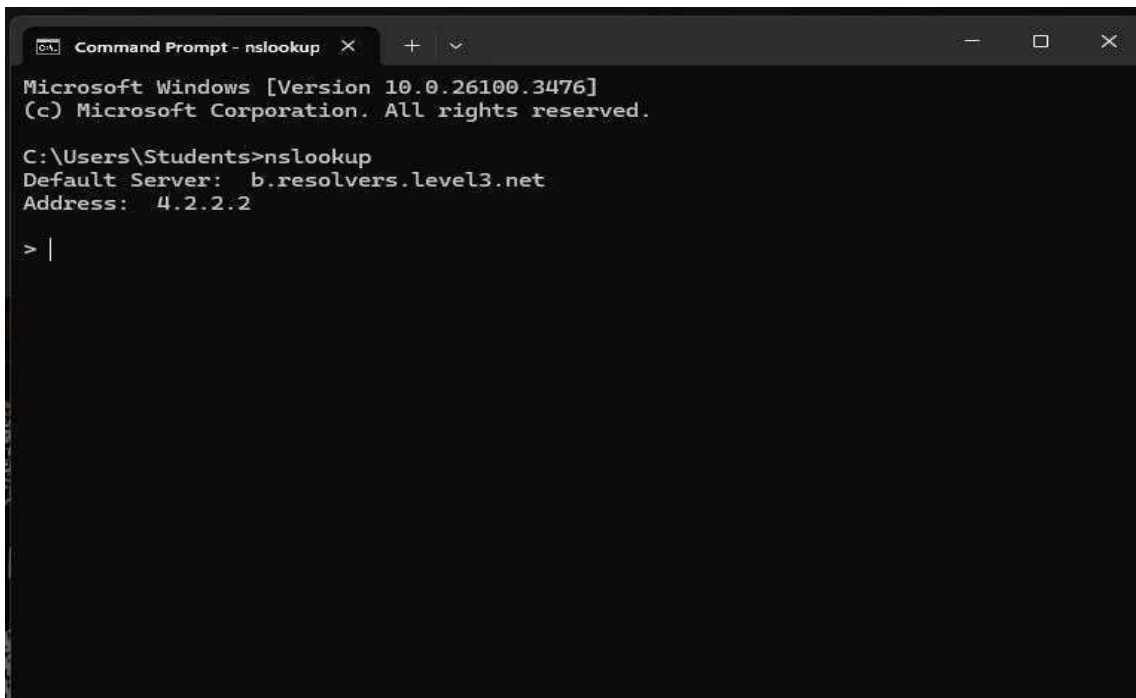
Windows IP Configuration

Ethernet adapter Ethernet:

    Connection-specific DNS Suffix  . : 
    Link-local IPv6 Address . . . . . : fe80::c6ef:a94b:74ad:f010%5
    IPv4 Address. . . . . : 172.16.108.201
    Subnet Mask . . . . . : 255.255.0.0
    Default Gateway . . . . . : 172.16.105.1

C:\Users\Students>
```

3. NSLOOKUP

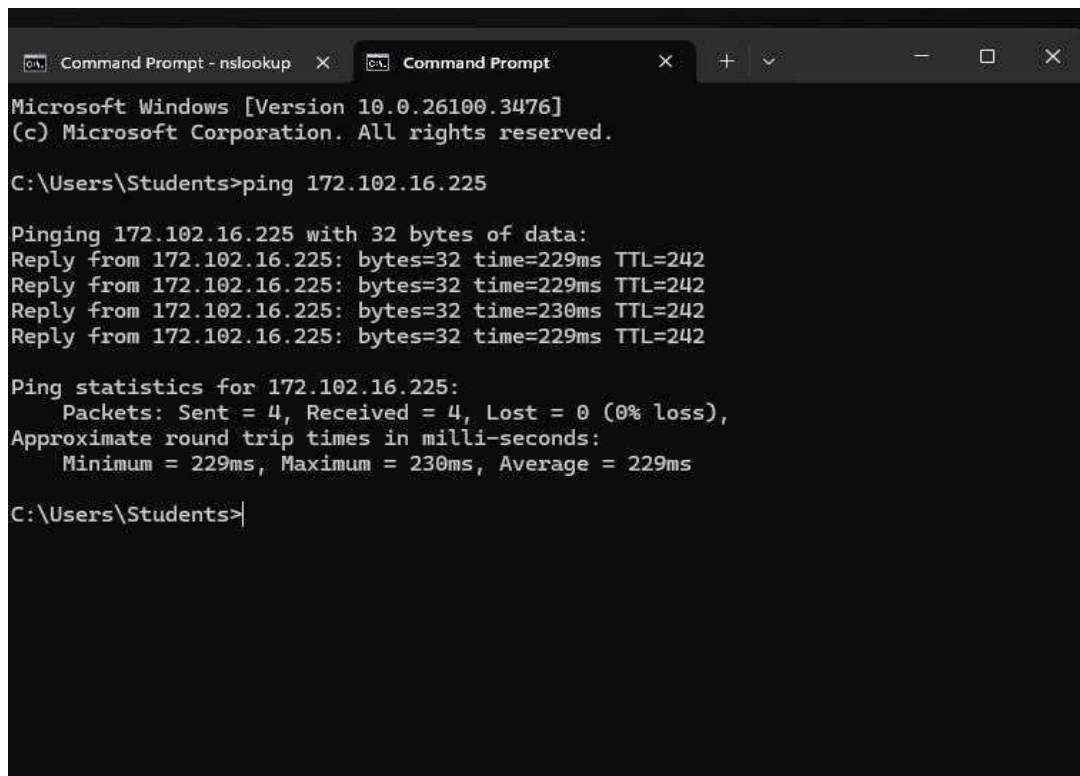


```
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Students>nslookup
Default Server:  b.resolvers.level3.net
Address:  4.2.2.2

> |
```

4. PING



```
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Students>ping 172.102.16.225

Pinging 172.102.16.225 with 32 bytes of data:
Reply from 172.102.16.225: bytes=32 time=229ms TTL=242
Reply from 172.102.16.225: bytes=32 time=229ms TTL=242
Reply from 172.102.16.225: bytes=32 time=230ms TTL=242
Reply from 172.102.16.225: bytes=32 time=229ms TTL=242

Ping statistics for 172.102.16.225:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 229ms, Maximum = 230ms, Average = 229ms

C:\Users\Students>|
```

5. Traceroute

```
Command Prompt
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Students>tracert www.google.com

Tracing route to www.google.com [142.250.187.228]
over a maximum of 30 hops:

  1  <1 ms    <1 ms    <1 ms    172.16.105.1
  2  1 ms     1 ms     <1 ms    47.247.2.81
  3  1 ms     1 ms     1 ms     139.167.57.185
  4  *         *         *         Request timed out.
  5  4 ms     4 ms     5 ms     172.16.12.122
  6  3 ms     3 ms     3 ms     172.16.12.122
  7  6 ms     5 ms     5 ms     72.14.217.252
  8  3 ms     2 ms     2 ms     216.239.43.131
  9  3 ms     3 ms     3 ms     142.251.50.58
 10 36 ms    36 ms    37 ms    142.251.230.221
 11 64 ms    64 ms    67 ms    192.178.108.38
 12 194 ms   194 ms   194 ms    216.239.48.246
 13 *       *       *         Request timed out.
 14 236 ms   *       *         142.251.229.31
 15 *       245 ms *         216.239.49.152
 16 260 ms   264 ms  261 ms    142.251.69.70
 17 329 ms   330 ms  338 ms    142.251.69.73
 18 330 ms   329 ms  329 ms    142.251.233.54
 19 329 ms   329 ms  330 ms    192.178.97.171
 20 328 ms   328 ms  329 ms    142.251.54.47
 21 328 ms   328 ms  328 ms    lhr25s34-in-f4.1e100.net [142.250.187.228]

Trace complete.

C:\Users\Students>
```

Result:

Thus the various commands like tcpdump, Ipconfig, netstat, ping are executed successfully.

EX NO.2**CREATE A HTTP WEB CLIENT PROGRAM TO
DOWNLOAD A WEB PAGE USING TCP SOCKETS****AIM:**

To write a java program for socket for HTTP for web page to download.

ALGORITHM:

1. Start the program.
2. Get the frame size from the user
3. To create the frame based on the user request.
4. To send frames to server from the client side.
5. If your frames reach the server it will send ACK signal to client otherwise it will send NACK signal to client.
6. Stop the program

PROGRAM:**CLIENT**

```
import javax.swing.*;
import java.net.*;
import java.awt.image.*;
import javax.imageio.*;
import java.io.*;
import java.awt.image.BufferedImage;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.IOException; import
javax.imageio.ImageIO;
public class Client{
    public static void main(String args[]) throws Exception{
        Socket soc;
        BufferedImage img = null;
        soc=new Socket("localhost",4000);
        System.out.println("Client is running. ");
        Try
        {
            System.out.println("Reading image from disk. ");
            img = ImageIO.read(new File("digital_image_processing.jpg"));
            ByteArrayOutputStream baos = new ByteArrayOutputStream();
            ImageIO.write(img, "jpg", baos);
            baos.flush();
            byte[] bytes = baos.toByteArray();baos.close();
            System.out.println("Sending image to server. ");
            OutputStream out = soc.getOutputStream();
            DataOutputStream dos = new DataOutputStream(out);
            dos.writeInt(bytes.length);
            dos.write(bytes, 0, bytes.length);
            System.out.println("Image sent to server. ");
            dos.close();
```

```

out.close();
}
catch (Exception e) { System.out.println("Exception: " + e.getMessage());
soc.close();
}
soc.close();
}
}

```

SERVER

```

import java.net.*;
import java.io.*;
import java.awt.image.*;
import javax.imageio.*;
import javax.swing.*;
class Server
{
public static void main(String args[]) throws Exception {
ServerSocket server=null;
Socket socket;
server=new ServerSocket(4000);
System.out.println("Server Waiting for image");
socket=server.accept(); System.out.println("Client connected.");
InputStream in = socket.getInputStream();
DataInputStream dis = new DataInputStream(in);
int len = dis.readInt();
System.out.println("Image Size: " + len/1024 + "KB"); byte[] data = new byte[len];
dis.readFully(data);
dis.close();
in.close();
InputStream ian = new ByteArrayInputStream(data);
BufferedImage bImage = ImageIO.read(ian);
JFrame f = new JFrame("Server");
ImageIcon icon = new ImageIcon(bImage);
JLabel l = new JLabel();
l.setIcon(icon);
f.add(l);
f.pack();
f.setVisible(true);
}
}

```

OUTPUT

When you run the client code, following output screen would appear on client side.

```
Microsoft Windows [Version 10.0.22631.5189]
(c) Microsoft Corporation. All rights reserved.

C:\Users\New>d:

D:\>javac Client.java

D:\>java Client digital_image_processing.jpg
Client is running.
Reading image from disk.
Sending image to server.
Image sent to server.

D:\>|
```

```
Microsoft Windows [Version 10.0.22631.5189]
(c) Microsoft Corporation. All rights reserved.

C:\Users\New>d:

D:\>javac Server.java

D:\>java Server
Server Waiting for image
Client connected.
Image Size: 264KB
|
```



RESULT: Thus the program for HTTP web client program to download a web page using TCP socket was executed and the output is verified successfully.

EXNO: 3A

**APPLICATIONS USING TCP SOCKETS
IMPLEMENTATION OF ECHO**

AIM:

To Write a Network Socket program for implementing the echo operation.

ALGORITHM:

TCP Echo Server

1. Include necessary header files to support functions for Socket definitions,Socket types,Internet addresses,I/O functions,UNIX system calls.
2. Declare variables for Socket ID,Port number,Socket addresses,Character buffer etc.
3. Create socket for server and bind socket with addresses.
4. Specify number of allowed connections.
5. Wait for connection.
6. Accept connection if any.
7. Retrieve information from Connected Socket.
8. Echo retrieved information to Connected Socket.
9. Close Connected Socket.

TCP Echo Client

1. Include necessary header files to support functions for Socket definitions,Socket types,Internet addresses,I/O functions,UNIX system calls.
2. Declare variables for Socket ID,Port number,Socket addresses,Character buffer etc.
3. Create socket for client.
4. Connect client socket to the server socket addresses.
5. Give message to be echoed.
6. Retrieve echoed information fromserver and display it.

SERVER PROGRAM:

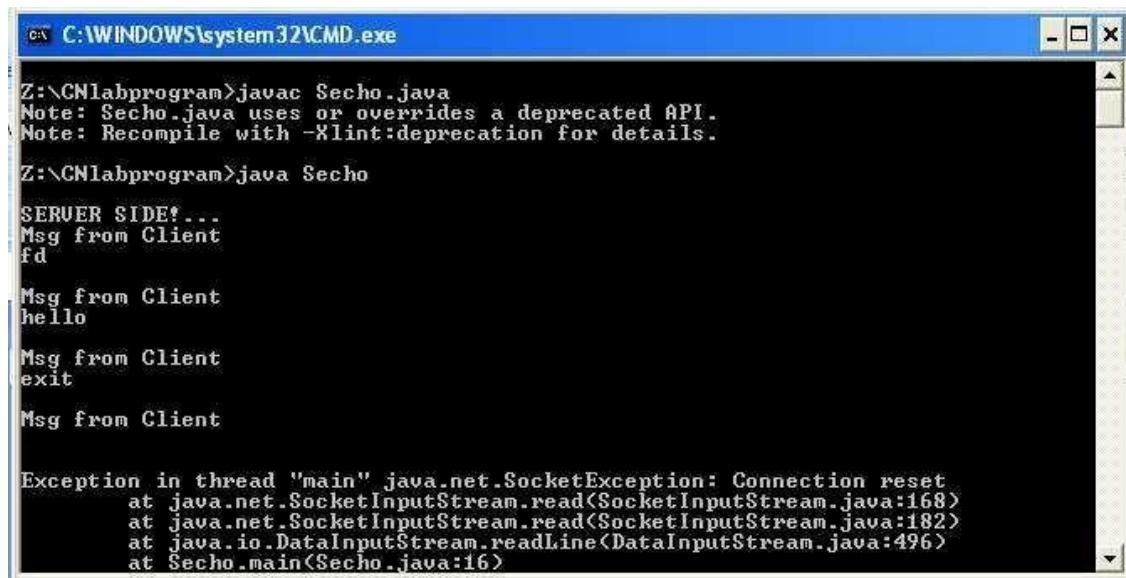
```
import java.io.*;
import java.net.*;
import java.lang.String.*;
public class Secho
{
public static void main(String args[]) throws Exception
{
ServerSocket ss =new ServerSocket(123);
Socket s=ss.accept();
DataInputStream in= new DataInputStream(s.getInputStream());
DataOutputStream out=new DataOutputStream(s.getOutputStream());
String str;
System.out.println("\nSERVER SIDE!...");
while(true)
{
str=in.readLine();
out.writeBytes(str+"\n");
System.out.println("Msg from Client");
System.out.println(str+"\n");
}
}
}
```

CLIENT PROGRAM:

```
import java.io.*;
import java.net.*;
import java.lang.String.*;
public class Cecho
{
public static void main(String args[]) throws Exception
{
DataInputStream in=new DataInputStream (System.in);
Socket s=new Socket("LocalHost",123);
DataInputStream inecho=new DataInputStream(s.getInputStream());
DataOutputStream out=new DataOutputStream(s.getOutputStream());
String str;
System.out.println("\nCLIENT SIDE!...\nEnter Client Msg");
while((str=in.readLine())!=null)
{
out.writeBytes(str+"\n");
if(str.equals("exit"))
{
out.writeBytes("\nClient Terminated");
break;
}
else
{
System.out.println("\nEcho From Server");
System.out.print(str+"\n");
System.out.println("\nCLIENT SIDE!...\nEnter Client Msg");
}
}
}
}
```


OUTPUT

SERVER:



```
C:\WINDOWS\system32\CMD.exe

Z:\CNlabprogram>javac Secho.java
Note: Secho.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

Z:\CNlabprogram>java Secho

SERVER SIDE!...
Msg from Client
fd

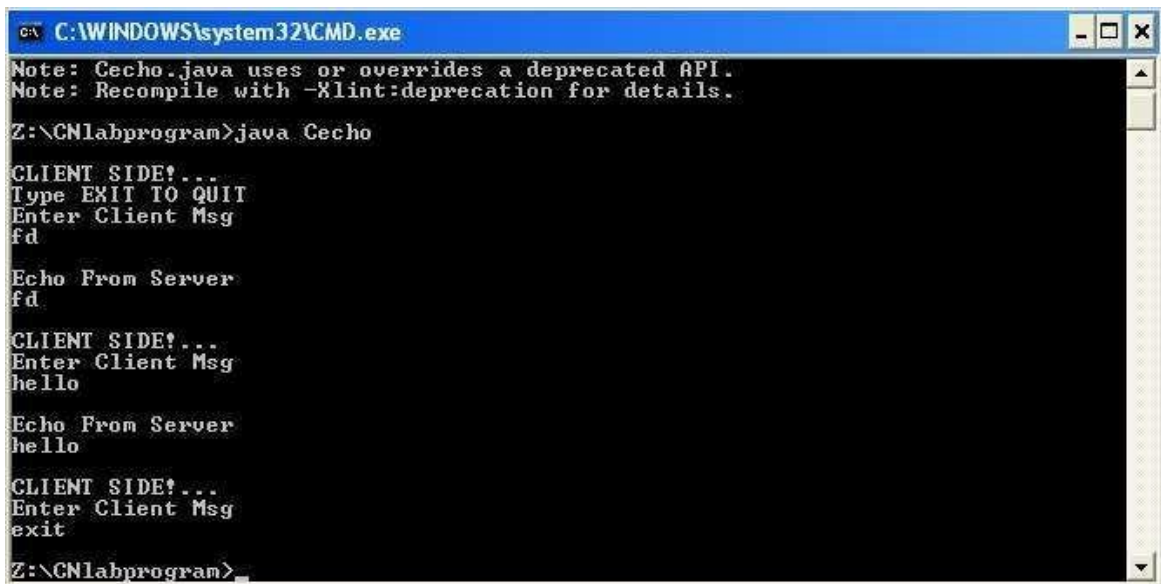
Msg from Client
hello

Msg from Client
exit

Msg from Client

Exception in thread "main" java.net.SocketException: Connection reset
    at java.net.SocketInputStream.read(SocketInputStream.java:168)
    at java.net.SocketInputStream.read(SocketInputStream.java:182)
    at java.io.DataInputStream.readLine(DataInputStream.java:496)
    at Secho.main(Secho.java:16)
```

CLIENT:



```
C:\WINDOWS\system32\CMD.exe

Note: Cecho.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

Z:\CNlabprogram>java Cecho

CLIENT SIDE!...
Type EXIT TO QUIT
Enter Client Msg
fd

Echo From Server
fd

CLIENT SIDE!...
Enter Client Msg
hello

Echo From Server
hello

CLIENT SIDE!...
Enter Client Msg
exit

Z:\CNlabprogram>
```

RESULT:

Thus the program for implementation of ECHO was executed and output is verified successfully.

EXNO: 3B**CHAT APPLICATION****AIM:**

To write a network program for implementing chat using TCP socket.

ALGORITHM:

- 1: Client sends the request to server
- 2: Server runs the request and the connection with the client that has requested the server.
- 3: The client sends the message to server.
- 4: The server process it and it is displayed (i.e.,) replier by sending the message to client also displayed.
- 5: Stop the program.

SERVER PROGRAM:

```
import java.io.*;
import java.net.*;
public class chatserver
{
    public static void main(String args[])throws Exception
    {
        DataInputStream din=null;
        DataOutputStream dout=null;
        Socket c=null;
        ServerSocket m=null;
        DataInputStream stdin=new DataInputStream(System.in);
        try
        {
            m=new ServerSocket(68);
            c=m.accept();
            din=new DataInputStream(c.getInputStream());
            dout=new DataOutputStream(c.getOutputStream());
        }
        catch(Exception e)
        {
        }
        while(c!=null)
        {
            String m2;
            System.out.println("Server");
            while(true)
            {
                String m1=din.readLine();
```

```

System.out.println("Message from client.."+m1);
System.out.println("\n\n Enter the message...");
m2=stdin.readLine();
dout.writeBytes(""+m2);
dout.writeBytes("\n");
}}
din.close();
dout.close();
c.close();
m.close();
}
}

```

CLIENT PROGRAM:

```

import java.io.*;
import java.net.*;
public class chatclient
{
    public static void main(String args[])throws Exception
    {
        Socket c=null;
        DataInputStream uin=null;
        DataInputStream din=null;
        DataOutputStream dout=null;
        try
        {
            c=new Socket("localhost",68);
            uin=new DataInputStream(System.in);
            din=new DataInputStream(c.getInputStream());
            dout=new DataOutputStream(c.getOutputStream());
        }
        catch(Exception e)
        {
        }
        if(c!=null)
        {
            String inp;
            System.out.println("Enter the message:");
            while((inp=uin.readLine())!=null)
            {
                dout.writeBytes(""+inp);
                dout.writeBytes("\n");
                System.out.println("Echoed message from server.."+din.readLine());
                System.out.println("Enter ur message:");
            }
            din.close();
            dout.close();
            c.close();
        }
    }
}

```

OUTPUT:

SERVER:



```
C:\WINDOWS\system32\CMD.exe - java chatserver

Z:\CNlabprogram>javac chatserver.java
Note: chatserver.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

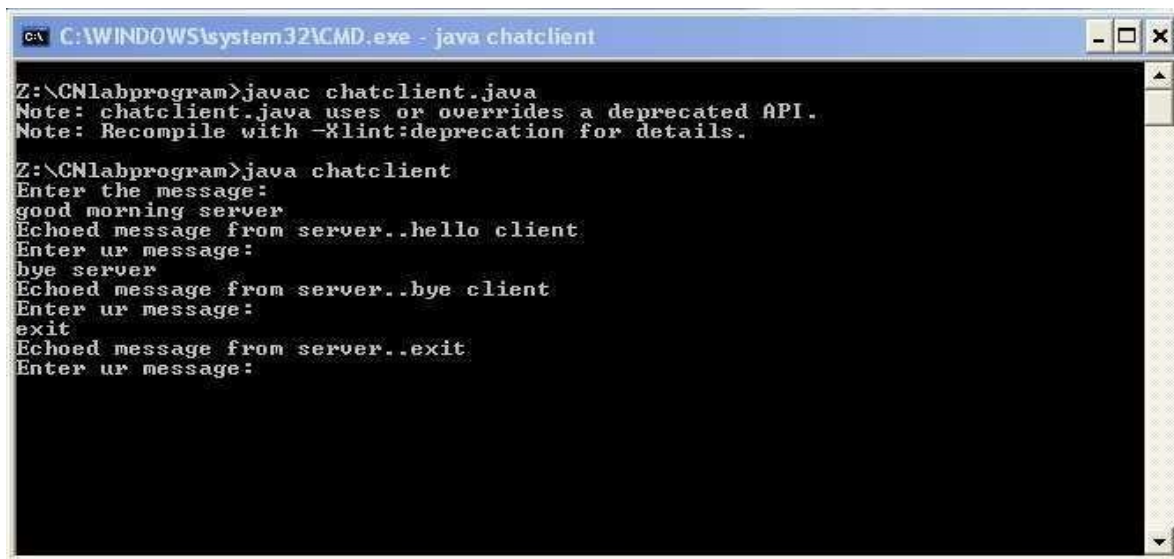
Z:\CNlabprogram>java chatserver
Server
Message from client..good morning server

Enter the message...
hello client
Message from client..bye server

Enter the message...
bye client
Message from client..exit

Enter the message...
exit
```

CLIENT:



```
C:\WINDOWS\system32\CMD.exe - java chatclient

Z:\CNlabprogram>javac chatclient.java
Note: chatclient.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

Z:\CNlabprogram>java chatclient
Enter the message:
good morning server
Echoed message from server..hello client
Enter ur message:
bye server
Echoed message from server..bye client
Enter ur message:
exit
Echoed message from server..exit
Enter ur message:
```

RESULT:

Thus the program for CHAT APPLICATION was executed and output is verified successfully.

EXNO: 3C

FILE TRANSFER

AIM:

To write a JAVA program to perform file transfer using TCP sockets

ALGORITHM:

SERVER:

1. Create a server socket
2. Create a new file
3. Get the input from the file using FileInputStream
4. Copy the contents of the file to the FileOutputStream
5. Close the socket
6. Stop the program

CLIENT:

1. Create a socket
2. Get the input from the socket using InputStream
3. Copy the contents of the file to the FileOutputStream
4. Print the contents of the file
5. Close the socket
6. Stop the program

PROGRAM

FILE CLIENT

```
import java.io.*;
import java.net.*;
import java.util.*;
class Clientfile
{
public static void main(String args[])
{
try
{
BufferedReader in=new BufferedReader(new InputStreamReader(System.in));
Socket clsct=new Socket("127.0.0.1",139);
DataInputStream din=new DataInputStream(clsct.getInputStream());
DataOutputStream dout=new DataOutputStream(clsct.getOutputStream());
System.out.println("Enter the file name:");
String str=in.readLine();
dout.writeBytes(str+"\n");
System.out.println("Enter the new file name:");
String str2=in.readLine();
```

```

String str1,ss;
FileWriter f=new FileWriter(str2);

char buffer[];
while(true)
{
str1=din.readLine();
if(str1.equals("-1")) break;
System.out.println(str1);
buffer=new char[str1.length()];
str1.getChars(0,str1.length(),buffer,0);
f.write(buffer);
}
f.close();
clsct.close();
}
catch (Exception e)
{
System.out.println(e);}}}

```

FILE SERVER

```

import java.io.*;
import java.net.*;
import java.util.*;
class Serverfile
{
public static void main(String args[])
{
try
{
ServerSocket obj=new ServerSocket(139);
while(true)
{
Socket obj1=obj.accept();
DataInputStream din=new DataInputStream(obj1.getInputStream());
DataOutputStream dout=new DataOutputStream(obj1.getOutputStream());
String str=din.readLine();
FileReader f=new FileReader(str);
BufferedReader b=new BufferedReader(f);
String s;
while((s=b.readLine())!=null)
{
System.out.println(s);
dout.writeBytes(s+"\n");
}
f.close();
dout.writeBytes("-1\n");
}
}
catch(Exception e)
{ System.out.println(e);}
}
}

```

OUTPUT:

```
C:\Program Files\Java\jdk1.5.0\bin>javac Clientfile.java
Note: Clientfile.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

C:\Program Files\Java\jdk1.5.0\bin>java Clientfile
Enter the file name:
Clientfile.java
Enter the new file name:

C:\Program Files\Java\jdk1.5.0\bin>javac Serverfile.java
Note: Serverfile.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

C:\Program Files\Java\jdk1.5.0\bin>java Serverfile
import java.io.*;
import java.net.*;
import java.util.*;
class Clientfile
{
    public static void main(String args[])
    {
        try
        {
            BufferedReader in=new BufferedReader(new InputStreamReader(System.in));
            Socket clscct=new Socket("127.0.0.1",139);
            DataInputStream din=new DataInputStream(clscct.getInputStream());
            DataOutputStream dout=new DataOutputStream(clscct.getOutputStream());
            System.out.println("Enter the file name:");
            String str=in.readLine();
            dout.writeBytes(str+'\n');
            System.out.println("Enter the new file name:");
            String str2=in.readLine();
            String str1,ss;
            FileWriter f=new FileWriter(str2);
            char buffer[];
            while(true)
            {
                str1=din.readLine();
                if(str1.equals("-1")) break;
                System.out.println(str1);
                buffer=new char[str1.length()+1];
                str1.getChars(0,str1.length(),buffer,0);
                f.write(buffer);
            }
            f.close();
            clscct.close();
        }
        catch (Exception e)
        {
            System.out.println(e);
        }
    }
}
```

RESULT:

Thus the program for FILE TRANSFER was executed and the output is verified successfully.

EXNO: 4**SIMULATION OF DNS USING UDP SOCKETS****AIM:**

To implement a DNS server and client in java using UDP sockets.

ALGORITHM:**SERVER**

1. Create an array of hosts and its ip address in another array
2. Create a datagram socket and bind it to a port
3. Create a datagram packet to receive client request
4. Read the domain name from client to be resolved
5. Lookup the host array for the domain name
6. If found then retrieve corresponding address
7. Create a datagram packet and send ip address to client
8. Repeat steps 3-7 to resolve further requests from clients
9. Close the server socket
10. Stop

CLIENT

1. Create a datagram socket
2. Get domain name from user
3. Create a datagram packet and send domain name to the server
4. Create a datagram packet to receive server message
5. Read server's response
6. If ip address then display it else display "Domain does not exist"
7. Close the client socket
8. Stop

PROGRAM

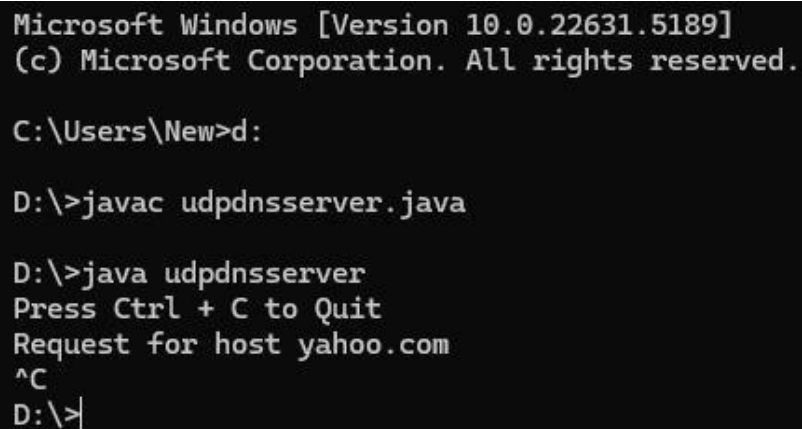
// UDP DNS Server -- udpdnsserver.java

```
import java.io.*;
import java.net.*;
public class udpdnsserver
{
    private static int indexOf(String[] array, String str)
    {
        str = str.trim();
        for (int i=0; i< array.length; i++)
        {
            if (array[i].equals(str))
                return i;
        }
        return -1;
    }
    public static void main(String arg[])throws IOException
    {
        String[] hosts = {"yahoo.com", "gmail.com", "cricinfo.com", "facebook.com"};
        String[] ip = {"68.180.206.184", "209.85.148.19", "80.168.92.140", "69.63.189.16"};
        System.out.println("Press Ctrl + C to Quit");
        while (true)
        {
            DatagramSocket serversocket=new DatagramSocket(1362);
            byte[] senddata = new byte[1021];
            byte[] receivedata = new byte[1021];
            DatagramPacket recvpack = new DatagramPacket(receivedata, receivedata.length);
            serversocket.receive(recvpack);
            String sen = new String(recvpack.getData());
            InetAddress ipaddress = recvpack.getAddress();
            int port = recvpack.getPort();
            String capsent;
            System.out.println("Request for host " + sen);
            if(indexOf (hosts, sen) != -1)
                capsent = ip[indexOf (hosts, sen)];
            else
                capsent = "Host Not Found";
            senddata = capsent.getBytes();
            DatagramPacket pack = new DatagramPacket(senddata, senddata.length, ipaddress, port);
            serversocket.send(pack);
            serversocket.close();
        }
    }
}
```

//UDP DNS Client -- udpdnscient.java

```
import java.io.*;
import java.net.*;
public class udpdnscient
{
    public static void main(String args[])throws IOException
    {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        DatagramSocket clientsocket = new DatagramSocket();
        InetAddress ipaddress;
        if (args.length == 0)
            ipaddress = InetAddress.getLocalHost();
        else
            ipaddress = InetAddress.getByName(args[0]);
        byte[] senddata = new byte[1024];
        byte[] receivedata = new byte[1024];
        int portaddr = 1362;
        System.out.print("Enter the hostname : ");
        String sentence = br.readLine();
        Senddata = sentence.getBytes();
        DatagramPacket pack = new DatagramPacket(senddata,senddata.length, ipaddress,portaddr);
        clientsocket.send(pack);
        DatagramPacket recvpack =new DatagramPacket(receivedata,receivedata.length);
        clientsocket.receive(recvpack);
        String modified = new String(recvpack.getData());
        System.out.println("IP Address: " + modified);
        clientsocket.close();
    }
}
```

OUTPUT



```
Microsoft Windows [Version 10.0.22631.5189]
(c) Microsoft Corporation. All rights reserved.

C:\Users\New>d:

D:\>javac udpdnsserver.java

D:\>java udpdnsserver
Press Ctrl + C to Quit
Request for host yahoo.com
^C
D:\>|
```

```
Microsoft Windows [Version 10.0.22631.5189]
(c) Microsoft Corporation. All rights reserved.

C:\Users\New>d:

D:\>javac udpdnsclient.java

D:\>java udpdnsclient
Enter the hostname : yahoo.com
IP Address: 68.180.206.184

D:\>|
```

RESULT:

Thus the program for implementation of DNS was executed and the output is verified successfully.

EXNO:5

Use a tool like Wireshark to capture packets and examine the packets.

AIM: To capture and examine network packets in Java, you would typically use a library that interacts with network interfaces.

ALGORITHM:

1. Install PCAP4J: First, you need to add the PCAP4J library to your Java project. You can
2. download it from the [PCAP4J GitHub page](#) or include it using Maven.
3. Launch Wireshark and select your network interface (e.g., eth0).
4. Start the capture by clicking the green shark fin.
5. Apply a capture filter to capture only TCP traffic on port 80 (HTTP):
6. Use the filter: tcp port 80.
7. After capturing traffic, you can apply a display filter to focus on the server's IP address:
8. Use the filter: ip.addr == 192.168.1.100.
9. Analyze packets by selecting individual packets and examining their details (IP addresses, n
10. protocols, HTTP request/response).
11. Stop the capture once you've gathered enough data.
12. Save the capture for later analysis or sharing by going to File -> Save As.

PROGRAM

```
import org.pcap4j.core.*;
import org.pcap4j.packet.*;
import org.pcap4j.util.*;
import java.net.InetAddress;
import java.util.List;
public class PacketCapture
{
public static void main(String[] args) throws PcapNativeException, NotOpenException
{
// Get the list of available network devices
List<PcapNetworkInterface> devices = Pcaps.findAllDevs();
if (devices.isEmpty())
{
System.out.println("No network devices found.");
return;}
}
```

```
// Select the first available device
PcapNetworkInterface device = devices.get(0);

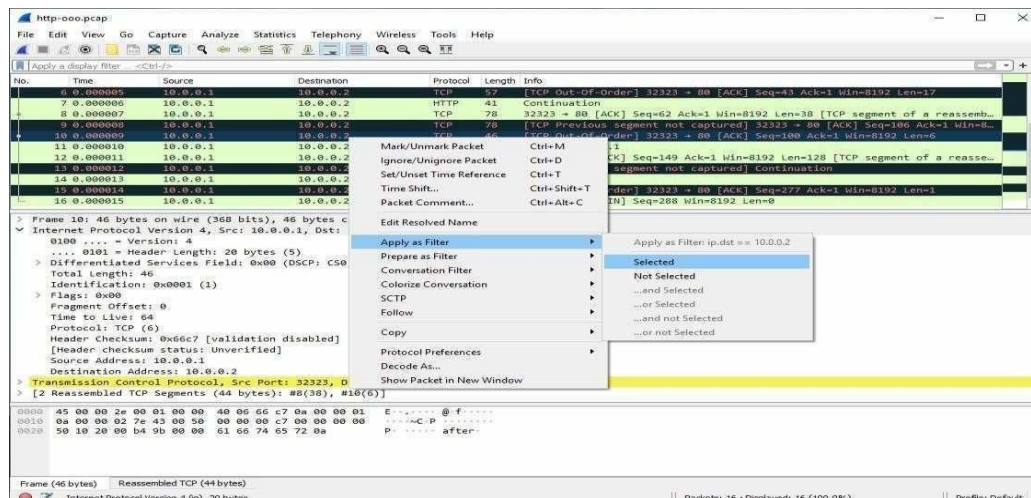
// Open the device with a buffer size of 10MB and capture live packets
PcapHandle handle = device.openLive(65536, PcapNetworkInterface.PromiscuousMode.PROMISCUOUS,
10);

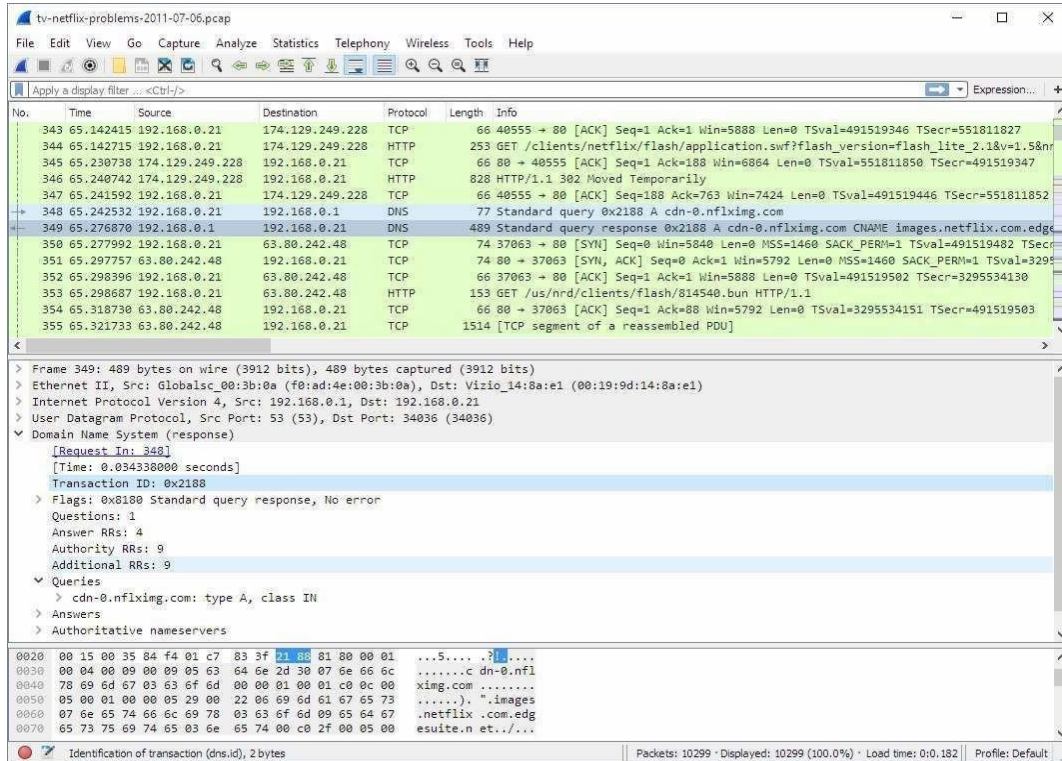
System.out.println("Capturing packets from: " + device.getName());

// Create a packet listener to print packet details
PacketListener listener = packet ->
{
System.out.println("Packet: " + packet);
if (packet.contains(IPPacket.class))
{
IPPacket ipPacket = packet.get(IPPacket.class);
InetAddress srcAddr = ipPacket.getHeader().getSrcAddr();
InetAddress dstAddr = ipPacket.getHeader().getDstAddr();
System.out.println("Source IP: " + srcAddr.getHostAddress());
System.out.println("Destination IP: " + dstAddr.getHostAddress());
}
};

// Start capturing packets and apply the listener
handle.loop(-1, listener);
}
```

OUTPUT:





Result:

Thus the program for capturing and examining of packets using wireshark has been executed successfully.

EXNO:6A**IMPLEMENTATION OF ARP PROTOCOLS****AIM:**

To write a java program for implementing the Address Resolution Protocol(ARP).

DESIGN APPROACH:

ARP is a low level protocol used to bind a address dynamically. It allows a host to find a physical address of a target host on the same physical network given only its IP address. ARP hides the underlying network physical addressing. It can be thought of as aprt of physical network system and not the internet protocols. ARP broadcasts special packets with destination's IP address to all hosts. The destination host only will respond with physical address. When the response is received, the sender ususe the physical address of destination host to send all the packets.

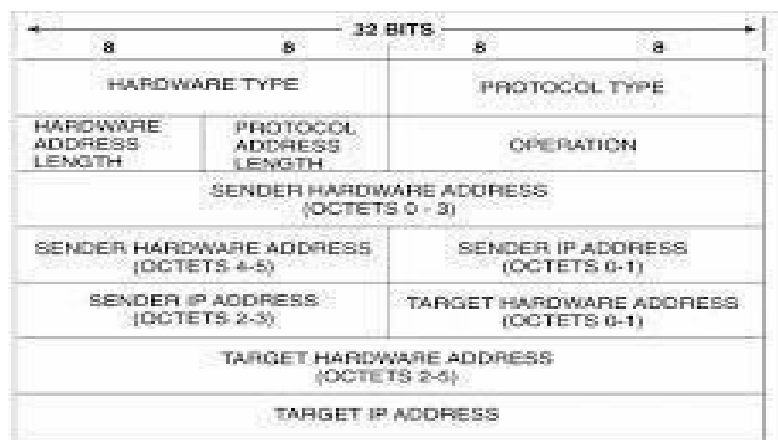
Hardware Type(16 bits): The type of interface the sender seeks an answer for. Protocol Type(16 bits): The high level software address to be provided.

HLEN(8 bits)- Length of the arbitrary physical address. PLEN(8 bits)-Length of the arbitrary protocol address

OPERATION(16 bits)-The specific type of operation required-
ARP.request-1, ARP.response-2

SENDER HA(6 octets)-The sender's actual hardware address scalable upto 6 bytes. SENDER IP(4 octets)-The sender's IP address, always 32 bits.

TARGET HA(6 octets)-The destination node's HA, scalable to 6 bytes. TARGET IP(4 octets)-The destination node's IP address,



always 32 bits.

FUNCTIONS USED:

1. FileInputStream- input stream that reads from a file
FileInputStream(String filename)
2. FileOutputStream-output stream that writes to a file
FileOutputStream(String filename)
3. Runtime.getRuntime()- This static method gives a reference to the current Runtime objects.
4. process exec(String program) throws IOException-executes the program specified by the program as a separate process
5. BufferedReader- Buffered Input character stream
6. InputStreamReader- input stream that translates bytes to character.
7. PrintWriter-Character based class for console output.
8. System.in-To read input from console.

Implementation of ARP

```
import java.io.*;
import java.net.*;
class Corarp
{
public static void main(String args[])throws Exception
{
try
{
BufferedReader in=null;
FileOutputStream fout;

FileInputStream fin;
Runtime r=Runtime.getRuntime();
Process p=r.exec("arp -a");
in=new BufferedReader(new InputStreamReader(p.getInputStream()));
fout=new FileOutputStream("out.data");
PrintWriter pw=new PrintWriter(fout,true);
String line;
while((line=in.readLine())!=null)
{
System.out.println(line);
pw.println(line);
}
fin=new FileInputStream("out.data");
BufferedReader binl=new BufferedReader(new InputStreamReader(System.in));
BufferedReader bin=new BufferedReader(new InputStreamReader(fin));
```



```

String l,l1,l2;
l1=bin1.readLine();
l1=" "+l1;
l=bin.readLine();
while((l=bin.readLine())!=null)
{
if(l.startsWith(l1))
{
l2=l.substring(25,41);
System.out.println(l2);
break;
}
}
in.close();
}
catch(IOException e){}
}
}

```

OUTPUT:

D:veltech >javac corarp.java
D:veltech >java corarp

Interface: 172.16.101.10 0*2.

INTERFACE ADDRESS	PHYSICAL ADDRESS	TYPE
172.16.98.101	00-15-58-d9-aa-7b	dynamic
172.16.98.101	00-01-6c-49-26-51	dynamic

172.16.98.101
 00-15-d9-aa-7b.

RESULT:

Thus the program to implementing ARP was executed and the output was verified successfully.

EXNO: 6B IMPLEMENTATION OF RARP PROTOCOLS

AIM:

To write a java program for implementing the Reverse Address Resolution Protocol (RARP)

DESIGN APPROACH:

RARP does the reverse operation of ARP, when we give the Physical address RARP provides the corresponding IP address

Hardware Type(16 bits): The type of interface the sender seeks an answer for.

Protocol Type(16 bits): The high level software address to be provided.

HLEN(8 bits)- Length of the arbitrary physical address.

PLEN(8 bits)-Length of the arbitrary protocol address

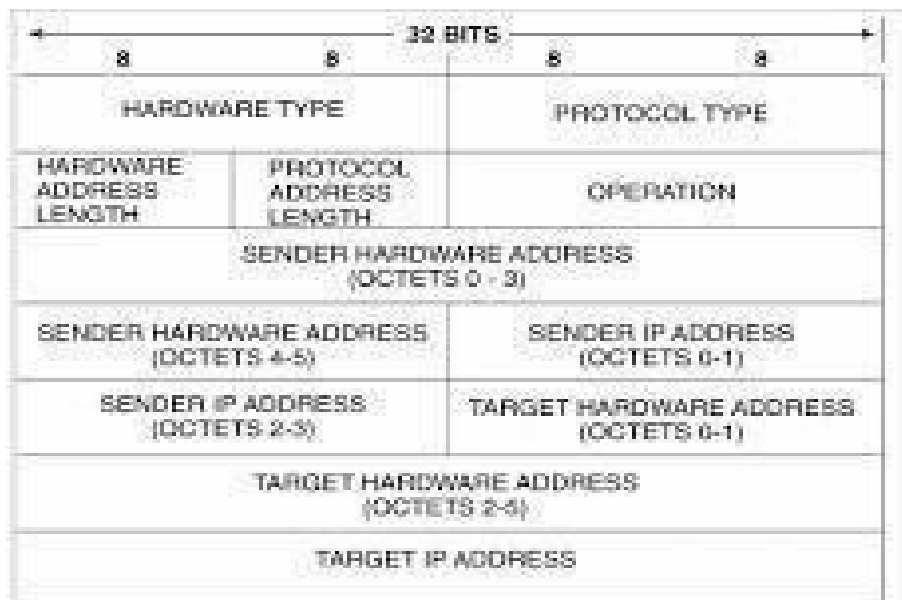
OPERATION(16 bits)-The specific type of operation required-ARP.request-1, ARP.response-2

SENDER HA(6 octets)-The sender's actual hardware address scalable upto 6 bytes.

SENDER IP(4 octets)-The sender's IP address, always 32 bits.

TARGET HA(6 octets)-The destination node's HA, scalable to 6 bytes.

TARGET IP(4 octets)-The destination node's IP address, always 32 bits.



FUNCTIONS USED:

1. FileInputStream- input stream that reads from a file FileInputStream(String filename)
2. FileOutputStream-output stream that writes to a file FileOutputStream(String filename)
3. Runtime.getRuntime()- This static method gives a reference to the current Runtime objects.
4. process exec(String program) throws IOException-executes the program specified by the program as a separate process
5. BufferedReader- Buffered Input character stream
6. InputStreamReader- input stream that translates bytes to character.
7. PrintWriter-Character based class for console output.
8. System.in-To read input from console.
9. String substring(int,int)- gives a part of string, starting from the position of first parameter, fill the position of the second parameter.
10. PrintWriterObject.println(string)- writes string to the file.

PROGRAM:

```
import java.io.*;
import java.net.*;
class Corrap
{
public static void main(String args[])throws Exception
{
try
{
BufferedReader in=null;
FileOutputStream fout;
FileInputStream fin;
Runtime r=Runtime.getRuntime();
Process p=r.exec("arp -a");
in=new BufferedReader(new InputStreamReader(p.getInputStream()));
fout=new FileOutputStream("out.data");
PrintWriter pw=new PrintWriter(fout,true);
String line;
while((line=in.readLine())!=null)
{
System.out.println(line);
pw.println(line);
}
}
```

```

fin=new FileInputStream("out.data");
BufferedReader bin1=new BufferedReader(new InputStreamReader(System.in));
BufferedReader bin=new BufferedReader(new InputStreamReader(fin));
String l,l1,l2;
l1=bin1.readLine();
l=bin.readLine();
a:while((l=bin.readLine())!=null)
{
if(l.startsWith("Int")||l.startsWith(" Int"))
{
continue a;
}
l2=l.substring(24,41);
if(l2.equals(l1))
{
System.out.println(l);
break;
}
}
in.close();
}
catch(IOException e){}
}
}

```

OUTPUT:

D:veltech>javac corrap.java

D:veltech>java corrap

Interface:172.16.101.10.....

INTERNET ADDRESS	PHYSICAL ADDRESS	TYPE
172.16.108.127	f8-bc-12-99-e7-c9	dynamic
f8-bc-12-99-e7-c9		
172.16.108.127	f8-bc-12-99-e7-c9	dynamic

RESULT:

Thus the program for implementing RARP was executed and the output is verified successfully.

AIM:

To study the network simulator and to simulate the congestion control mechanisms

PROCEDURE:

This section shows a simple NS simulation script and explains what each line does. Example 3 is an OTclscript that creates the simple network configuration and runs the simulation scenario

This network consists of 4 nodes (n0, n1, n2, n3) as shown in above figure. The duplex links between n0 and n2, and n1 and n2 have 2 Mbps of bandwidth and 10 ms of delay. The duplex link between n2 and n3 has 1.7 Mbps of bandwidth and 20 ms of delay. Each node uses a DropTail queue, of which the maximum size is 10. A "tcp" agent is attached to n0, and a connection is established to a tcp "sink" agent attached to n3. As default, the maximum size of a packet that a "tcp" agent can generate is 1KByte. A tcp "sink" agent generates and sends ACK packets to the sender (tcp agent) and frees the received packets. A "udp" agent that is attached to n1 is connected to a "null" agent attached to n3. A "null" agent just frees the packets received. A "ftp" and a "cbr" traffic generator are attached to "tcp" and "udp" agents respectively, and the "cbr" is configured to generate 1 KByte packets at the rate of 1 Mbps. The "cbr" is set to start at 0.1 sec and stop at 4.5 sec, and "ftp" is set to start at 1.0 sec and stop at sec.

The following is the explanation of the script above. In general, an NS script starts with making a Simulator object instance.

- set *ns* [new Simulator]: generates an NS simulator object instance, and assigns it to variable *ns* (italics is used for variables and values in this section). What this line does is the following:
 - Initialize the packet format (ignore this for now)
 - Create a scheduler (default is calendar scheduler)
 - Select the default address format (ignore this for now)

The "Simulator" object has member functions that do the following:

- Create compound objects such as nodes and links (described later)
 - Connect network component objects created (ex. attach-agent)
 - Set network component parameters (mostly for compound objects)
 - Create connections between agents (ex. make connection between a "tcp" and "sink")
 - Specify NAM display options etc.
- Most of member functions are for simulation setup (referred to as plumbing functions in the Overview section) and scheduling, however some of them are for the NAM display. The "Simulator" object member function implementations are located in the "ns-2/tcl/lib/ns-lib.tcl" file.

- *\$ns color fid color*: is to set color of the packets for a flow specified by the flow id (fid). This member function of "Simulator" object is for the NAM display, and has no effect on the actual simulation.
- *\$ns namtrace-all file-descriptor*: This member function tells the simulator to record simulation traces in NAM input format. It also gives the file name that the trace will be written to later by the command *\$ns flush-trace*. Similarly, the member function *trace-all* is for recording the simulation trace in a general format.
- *proc finish {}*: is called after this simulation is over by the command *\$ns at 5.0 "finish"*. In this function, post-simulation processes are specified.
- *set n0 [\$ns node]*: The member function *node* creates a node. A node in NS is compound object made of address and port classifiers (described in a later section). Users can create a node by separately creating an address and a port classifier objects and connecting them together. However, this member function of Simulator object makes the job easier. To see how a node is created, look at the files: "ns-2/tcl/libs/ns-lib.tcl" and "ns-2/tcl/libs/ns-node.tcl".
- *\$ns duplex-link node1 node2 bandwidth delay queue-type*: creates two simplex links of specified bandwidth and delay, and connects the two specified nodes. In NS, the output queue of a node is implemented as a part of a link, therefore users should specify the queue-type when creating links. In the above simulation script, DropTail queue is used. If the reader wants to use a RED queue, simply replace the word DropTail with RED. The NS implementation of a link is shown in a later section. Like a node, a link is a compound object, and users can create its sub-objects and connect them and the nodes. Link source codes can be found in "ns-2/tcl/libs/ns-lib.tcl" and "ns-2/tcl/libs/ns-link.tcl" files. One thing to note is that you can insert error modules in a link component to simulate a lossy link (actually users can make and insert any network objects). Refer to the NS documentation to find out how to do this.
- *\$ns queue-limit node1 node2 number*: This line sets the queue limit of the two simplex links that connect node1 and node2 to the number specified. At this point, the authors do not know how many of these kinds of member functions of Simulator objects are available and what they are. Please take a look at "ns-2/tcl/libs/ns-lib.tcl" and "ns-2/tcl/libs/ns-link.tcl", or NS documentation for more information.
- *\$ns duplex-link-op node1 node2 ...*: The next couple of lines are used for the NAM display. To see the effects of these lines, users can comment these lines out and try the simulation.

Now that the basic network setup is done, the next thing to do is to setup traffic agents such as TCP and UDP, traffic sources such as FTP and CBR, and attach them to nodes and agents respectively.

- *set tcp [new Agent/TCP]*: This line shows how to create a TCP agent. But in general, users can create anyagent or traffic sources in this way. Agents and traffic sources are in fact basic objects (not compound objects), mostly implemented in C++ and linked to OTcl. Therefore, there are no specific Simulator object member functions that create these object instances. To create agents or traffic sources, a user should know the class names these objects (Agent/TCP, Agent/TCPSink, Application/FTP and so on). This information can be found in the NS documentation or partly in this documentation. But one shortcut is to look at the "ns-2/tcl/libs/ns-default.tcl" file. This file contains the default configurable parameter value

settings for available network objects. Therefore, it works as a good indicator of what kind of network objects are available in NS and what are the configurable parameters.

- *\$ns attach-agent node agent*: The attach-agent member function attaches an agent object created to a node object. Actually, what this function does is call the attach member function of specified node, which attaches the given agent to itself. Therefore, a user can do the same thing by, for example, *\$n0 attach \$tcp*. Similarly, each agent object has a member function attach-agent that attaches a traffic source object to itself.
- *\$ns connect agent1 agent2*: After two agents that will communicate with each other are created, the next thing is to establish a logical network connection between them. This line establishes a network connection by setting the destination address to each others' network and port address pair.

Assuming that all the network configuration is done, the next thing to do is write a simulation scenario (i.e. simulation scheduling). The Simulator object has many scheduling member functions. However, the one that is mostly used is the following:

- *\$ns at time "string"*: This member function of a Simulator object makes the scheduler (scheduler_ is the variable that points the scheduler object created by [new Scheduler] command at the beginning of the script) to schedule the execution of the specified string at given simulation time. For example, *\$ns at 0.1 "\$cbr start"* will make the scheduler call a start member function of the CBR traffic source object, which starts the CBR to transmit data. In NS, usually a traffic source does not transmit actual data, but it notifies the underlying agent that it has some amount of data to transmit, and the agent, just knowing how much of the data to transfer, creates packets and sends them.

After all network configuration, scheduling and post-simulation procedure specifications are done, the only thing left is to run the simulation. This is done by *\$ns run*.

PROGRAM:

```
#Create a simulator object
set ns [new Simulator]

#Define different colors for data flows (for NAM)
$ns color 1 Blue
$ns color 2 Red

#Open the NAM trace file
set nf [open out.nam w]
$ns namtrace-all $nf

#Define a 'finish' procedure
proc finish {} {
    global ns nf
    $ns flush-trace
    #Close the NAM trace file
    close $nf
    #Execute NAM on the trace file
    exec nam out.nam &
    exit 0
}

#Create four nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]

#Create links between the nodes
$ns duplex-link $n0 $n2 2Mb 10ms DropTail
$ns duplex-link $n1 $n2 2Mb 10ms DropTail
$ns duplex-link $n2 $n3 1.7Mb 20ms DropTail

#Set Queue Size of link (n2-n3) to 10
$ns queue-limit $n2 $n3 10

#Give node position (for NAM)
$ns duplex-link-op $n0 $n2 orient right-down
$ns duplex-link-op $n1 $n2 orient right-up
$ns duplex-link-op $n2 $n3 orient right

#Monitor the queue for link (n2-n3). (for NAM)
$ns duplex-link-op $n2 $n3 queuePos 0.5
#Setup a TCP connection
set tcp [new Agent/TCP]
$tcp set class_ 2
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink]
$ns attach-agent $n3 $sink
```



```

$ns connect $tcp $sink
$tcp set fid_ 1

#Setup a FTP over TCP connection
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP

#Setup a UDP connection
set udp [new Agent/UDP]
$ns attach-agent $n1 $udp
set null [new Agent/Null]
$ns attach-agent $n3 $null
$ns connect $udp $null
$udp set fid_ 2

#Setup a CBR over UDP connection
set cbr [new Application/Traffic/CBR]
$cbr attach-agent $udp
$cbr set type_ CBR
$cbr set packet_size_ 1000
$cbr set rate_ 1mb
$cbr set random_ false

#Schedule events for the CBR and FTP agents
$ns at 0.1 "$cbr start"
$ns at 1.0 "$ftp start"
$ns at 4.0 "$ftp stop"
$ns at 4.5 "$cbr stop"

#Detach tcp and sink agents (not really necessary)
$ns at 4.5 "$ns detach-agent $n0 $tcp ; $ns detach-agent $n3 $sink"

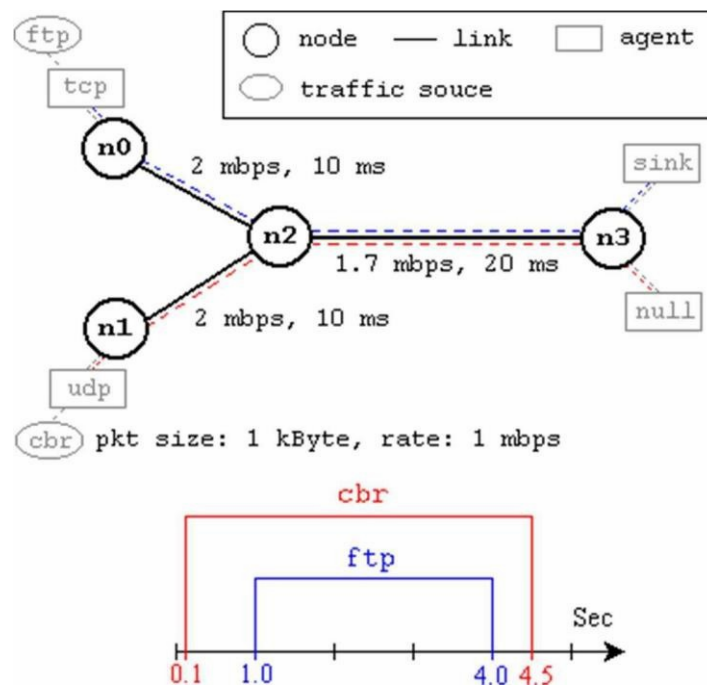
#Call the finish procedure after 5 seconds of simulation time
$ns at 5.0 "finish"

#Print CBR packet size and interval
puts "CBR packet size = [$cbr set packet_size_]"
puts "CBR interval = [$cbr set interval_]"

#Run the simulation
$ns run

```

OUTPUT



RESULT:

Thus the NS2 simulation for congestion control was successfully completed

EXNO: 8A**STUDY OF TCP PERFORMANCE****AIM:**

To simulate the TCP Performance using NS2

ALGORITHM:

1. Create a simulator object
2. Define different flows for data flows
3. Trace all events in a nam file and text file
4. Create source nodes ($s1$, $s2$, $s3$), gateway (G) and receiver (r)
5. Describe their layout topology
6. Specify the link between nodes
7. Define the queue size between nodes G and r as 5
8. Monitor queue on all links vertically 90°
9. Create TCP agents $tcp1$, $tcp2$, $tcp3$ and attach it to nodes $s1$, $s2$ and $s3$ respectively
10. Create three TCP sinks and attach it to node r
11. Connect traffic sources to the sink
12. Create FTP agents $ftp1$, $ftp2$, $ftp3$ and attach it to $tcp1$, $tcp2$ and $tcp3$ respectively
13. Label the nodes at start time
14. Schedule $ftp1$, $ftp2$, $ftp3$ to start at 0.1 and stop at 5.0seconds
15. Call *finish* procedure at 5.25 seconds
16. Run the simulation
17. Execute NAM on the trace file
18. Observe the simulated events on the NAM editor and packet flow on link G to r
19. View the trace file and analyse the events
20. Stop

PROGRAM:

```
#Study of TCP performance -TCP.tcl
```

```
#Create a simulator object
```

```
set ns [new Simulator]
```

```
#Open trace files
```

```
set f [open droptail-queue-out.tr w]
```

```
$ns trace-all $f
```

```
#Open the nam trace file
```

```
set nf [open droptail-queue-out.nam w]
```

```
$ns namtrace-all $nf
```

```
#s1, s2 and s3 act as sources. set s1 [$ns node]
```

```
set s2 [$ns node] set s3 [$ns node]
```

```
#G acts as a gateway set G [$ns node]
```

```
#r acts as a receiver set r [$ns node]
```

```
#Define different colors for data flows
```

```
$ns color 1 red
```

```
$ns color 2 SeaGreen
```

```
$ns color 3 blue
```

```
#Create links between the nodes
```

```
$ns duplex-link $s1 $G 6Mb 10ms DropTail
```

```
$ns duplex-link $s2 $G 6Mb 10ms DropTail
```

```
$ns duplex-link $s3 $G 6Mb 10ms DropTail
```

```
$ns duplex-link $G $r 3Mb 10ms DropTail
```

```
#Define the layout of the nodes
```

```
$ns duplex-link-op $s1 $G orient right-up
```

```
$ns duplex-link-op $s2 $G orient right
```

```
$ns duplex-link-op $s3 $G orient right-down
```

```
$ns duplex-link-op $G $r orient right
```

```
#Define the queue size for the link between node G and r
```

```
$ns queue-limit $G $r 5
```

```
#Monitor the queues for links vertically
```

```
$ns duplex-link-op $s1 $G queuePos 0.5
```

```
$ns duplex-link-op $s2 $G queuePos 0.5
```

```
$ns duplex-link-op $s3 $G queuePos 0.5
```

```
$ns duplex-link-op $G $r queuePos 0.5
```

```
#Create a TCP agent and attach it to node s1 set tcp1 [new Agent/TCP/Reno]
```

```
$ns attach-agent $s1 $tcp1
```

```
$tcp1 set window_ 8
```

```
$tcp1 set fid_1
```

```

#Create a TCP agent and attach it to node s2 set tcp2 [new Agent/TCP/Reno]
$ns attach-agent $s2 $tcp2
$tcp2 set window_ 8
$tcp2 set fid_2

#Create a TCP agent and attach it to node s3 set tcp3 [new Agent/TCP/Reno]
$ns attach-agent $s3 $tcp3
$tcp3 set window_ 4
$tcp3 set fid_3

#Create TCP sink agents and attach them to node r
set sink1 [new Agent/TCPSink]
set sink2 [new Agent/TCPSink]
set sink3 [new Agent/TCPSink]
$ns attach-agent $r $sink1
$ns attach-agent $r $sink2
$ns attach-agent $r $sink3

#Connect the traffic sources with the traffic sinks
$ns connect $tcp1 $sink1
$ns connect $tcp2 $sink2
$ns connect $tcp3 $sink3

#Create FTP applications and attach them to agents
set ftp1 [new Application/FTP]
$ftp1 attach-agent $tcp1
set ftp2 [new Application/FTP]
$ftp2 attach-agent $tcp2
set ftp3 [new Application/FTP]
$ftp3 attach-agent $tcp3

#Define a 'finish' procedure proc finish {}
{
global ns
$ns flush-trace
puts "running nam. "
exec nam -a droptail-queue-out.nam & exit 0
}

#Define label for nodes
$ns at 0.0 "$s1 label Sender1"
$ns at 0.0 "$s2 label Sender2"
$ns at 0.0 "$s3 label Sender3"
$ns at 0.0 "$G label Gateway"
$ns at 0.0 "$r label Receiver"

#Schedule ftp events
$ns at 0.1 "$ftp1 start"
$ns at 0.1 "$ftp2 start"
$ns at 0.1 "$ftp3 start"
$ns at 5.0 "$ftp1 stop"
$ns at 5.0 "$ftp2 stop"
$ns at 5.0 "$ftp3 stop"

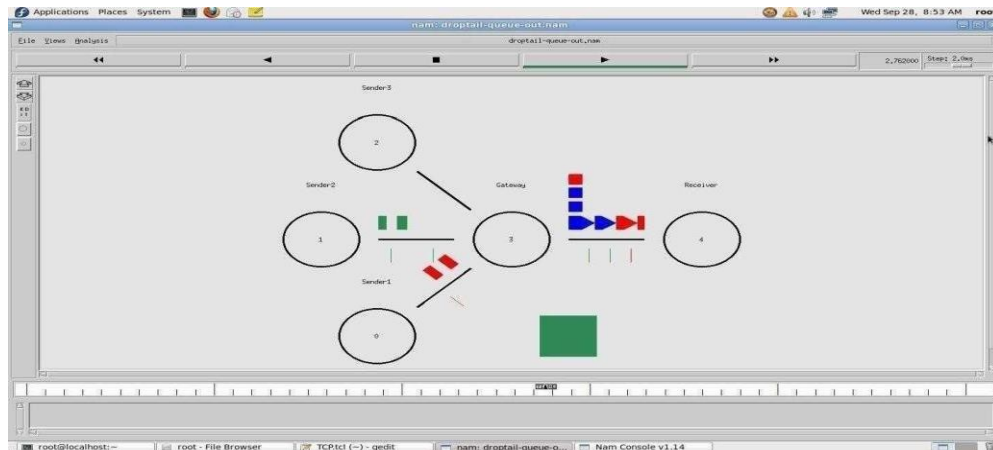
```

#Call finish procedure after 5 seconds of simulation time
\$ns at 5.25 "finish"

#Run the simulation
\$ns run

OUTPUT

\$ ns TCP.tcl



The screenshot shows the NS2 command-line interface with the output of the simulation. The output displays a series of TCP and UDP packets being sent and received, along with their sequence numbers and timestamps. The interface includes a menu bar (File, Edit, View, Search, Tools, Documents, Help) and a status bar at the bottom showing the current time as 17.228 and the step as 2.15s.

```
r 0.148427 3 1 ack 48 ----- 2 4.1 1.8 8 4
+ 0.148427 1 3 tcp 1040 ----- 2 1.0 4.1 1 8
- 0.148427 1 3 tcp 1040 ----- 2 1.0 4.1 1 8
+ 0.148427 1 3 tcp 1040 ----- 2 1.0 4.1 2 9
r 0.148533 3 2 ack 40 ----- 3 4.2 2.0 0 5
+ 0.148533 2 3 tcp 1040 ----- 3 2.0 4.2 1 10
- 0.148533 2 3 tcp 1040 ----- 3 2.0 4.2 1 10
+ 0.148533 2 3 tcp 1040 ----- 3 2.0 4.2 2 11
- 0.141707 0 3 tcp 1040 ----- 1 0.0 4.0 2 7
- 0.141813 1 3 tcp 1040 ----- 2 1.0 4.1 2 9
- 0.14192 2 3 tcp 1040 ----- 3 2.0 4.2 2 11
r 0.151707 0 3 tcp 1040 ----- 1 0.0 4.0 1 6
+ 0.151707 3 4 tcp 1040 ----- 1 0.0 4.0 1 6
- 0.151707 3 4 tcp 1040 ----- 1 0.0 4.0 1 6
r 0.151813 1 3 tcp 1040 ----- 2 1.0 4.1 1 8
+ 0.151813 3 4 tcp 1040 ----- 2 1.0 4.1 1 8
r 0.15192 2 3 tcp 1040 ----- 3 2.0 4.2 1 10
+ 0.15192 3 4 tcp 1040 ----- 3 2.0 4.2 1 10
r 0.153093 0 3 tcp 1040 ----- 1 0.0 4.0 2 7
+ 0.153093 3 4 tcp 1040 ----- 1 0.0 4.0 2 7
r 0.1532 1 3 tcp 1040 ----- 2 1.0 4.1 2 9
+ 0.1532 3 4 tcp 1040 ----- 2 1.0 4.1 2 9
r 0.153307 2 3 tcp 1040 ----- 3 2.0 4.2 2 11
+ 0.153307 3 4 tcp 1040 ----- 3 2.0 4.2 2 11
d 0.153307 3 4 tcp 1040 ----- 3 2.0 4.2 2 11
- 0.15448 3 4 tcp 1040 ----- 2 1.0 4.1 1 8
- 0.157253 3 4 tcp 1040 ----- 3 2.0 4.2 1 10
- 0.160027 3 4 tcp 1040 ----- 1 0.0 4.0 2 7
- 0.1628 3 4 tcp 1040 ----- 2 1.0 4.1 2 9
r 0.16448 3 4 tcp 1040 ----- 1 0.0 4.0 1 6
+ 0.16448 4 3 ack 40 ----- 1 4.0 0.0 1 12
- 0.16448 4 3 ack 40 ----- 1 4.0 0.0 1 12
r 0.167253 3 4 tcp 1040 ----- 2 1.0 4.1 1 8
+ 0.167253 4 3 ack 40 ----- 2 4.1 1.0 1 13
- 0.167253 4 3 ack 40 ----- 2 4.1 1.0 1 13
r 0.170027 3 4 tcp 1040 ----- 3 2.0 4.2 1 10
+ 0.170027 4 3 ack 40 ----- 3 4.2 2.0 1 14
- 0.170027 4 3 ack 40 ----- 3 4.2 2.0 1 14
r 0.1728 3 4 tcp 1040 ----- 1 0.0 4.0 2 7
+ 0.1728 4 3 ack 40 ----- 1 4.0 0.0 2 15
- 0.1728 4 3 ack 40 ----- 1 4.0 0.0 2 15
```

RESULT:

Thus the simulation of TCP performance was done using NS2 successfully

AIM:

To study the performance of UDP by simulating a simple network

ALGORITHM:

1. Create a simulator object
2. Define different color for data flows
3. Trace all events in a nam file.
4. Create four nodes $n0$, $n1$, $n2$ and $n3$
5. Describe their layout topology
6. Specify the link capacity between nodes
7. Monitor queue on the link $n2$ to $n3$ vertically 90°
8. Create a UDP agents $udp0$, $udp1$ and attach it to nodes $n0$ and $n1$ respectively
9. Create a CBR traffic $cbr0$, $cbr1$ and attach it to $udp0$ and $udp1$ respectively
10. Create a traffic sink and attach it to node $n3$
11. Connect sources to the sink
12. Label the nodes
13. Schedule $cbr0$ to start at 0.5 and stop at 4.5 seconds
14. Schedule $cbr1$ to start at 1.0 and stop at 4.0 seconds
15. Call finish procedure at 5.0 seconds
16. Run the simulation
17. Execute NAM on the trace file
18. Observe simulated events on the NAM and packet flow on link $n2$ to $n3$
19. Stop

PROGRAM

```
#Study of UDP performance - UDP.tcl

#Create a simulator object
set ns [new Simulator]

#Define different colors for data flows
$ns color 1 Blue
$ns color 2 Red

#Open the nam trace file set nf[open out.nam w]
$ns namtrace-all $nf

#Create four nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]

#Create links between the nodes
$ns duplex-link $n0 $n2 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n3 $n2 1Mb 10ms SFQ

#Specify layout of nodes
$ns duplex-link-op $n0 $n2 orient right-down
$ns duplex-link-op $n1 $n2 orient right-up
$ns duplex-link-op $n2 $n3 orient right

#Monitor the queue for the link 2—3 vertically
$ns duplex-link-op $n2 $n3 queuePos 0.5

#Create a UDP agent and attach it to node n0
set udp0 [new Agent/UDP]
$udp0 set class_ 1
$ns attach-agent $n0 $udp0

# Create a CBR traffic source and attach it to udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0

#Create a UDP agent and attach it to node n1 set udp1 [new Agent/UDP]
$udp1 set class_ 2
$ns attach-agent $n1 $udp1
```



```

# Create a CBR traffic source and attach it to udp1
set cbr1 [new Application/Traffic/CBR]
$cbr1 set packetSize_ 500
$cbr1 set interval_ 0.005
$cbr1 attach-agent $udp1

#Create a Null agent (a traffic sink) and attach it to node n3
set null0 [new Agent/Null]
$nns attach-agent $n3 $null0

#Connect traffic sources with the traffic sink
$nns connect $udp0 $null0
$nns connect $udp1 $null0

#Define finish procedure proc finish {}
{
    global ns nf
    $ns flush-trace

    #Close the trace file close $nf

    #Execute namon the trace file exec nam -a out.nam &
    exit 0
}

#Define label for nodes
$nns at 0.0 "$n0 label Sender1"
$nns at 0.0 "$n1 label Sender2"
$nns at 0.0 "$n2 label Router"
$nns at 0.0 "$n3 label Receiver"

#Schedule events for the CBR agents
$nns at 0.5 "$cbr0 start"
$nns at 1.0 "$cbr1 start"
$nns at 4.0 "$cbr1 stop"
$nns at 4.5 "$cbr0 stop"

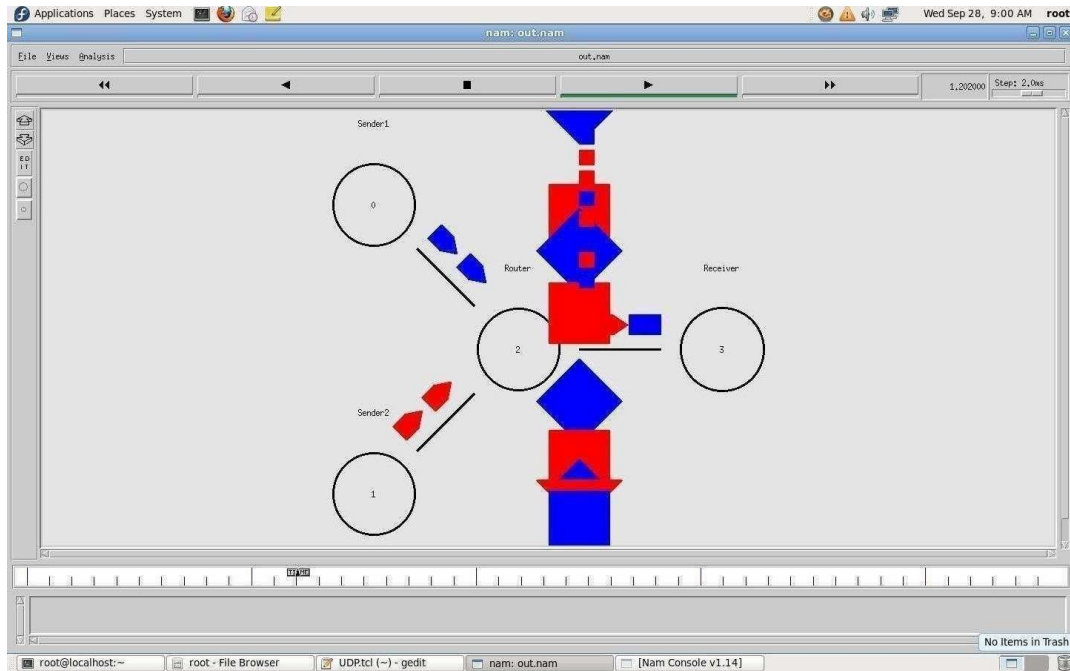
#Call finish procedure after 5 seconds of simulation time
$nns at 5.0 "finish"

#Run the simulation
$nns run

```

OUTPUT

\$ ns UDP.tcl



RESULT:

Thus the simulation of UDP performance was done using NS2 successfully

EXNO: 9A**SIMULATION OF DISTANCE VECTOR ROUTING****AIM:**

To simulate the distance vector routing algorithm using NS2

ALGORITHM:

1. Create a simulator object
2. Define different colors for different data flows
3. Open a namtrace file and define finish procedure then close the trace file, and execute nam on trace file.
4. Create n number of nodes using for loop
5. Create duplex links between the nodes
6. Setup UDP Connection between n(0) and n(5)
7. Setup another UDP connection between n(1) and n(5)
8. Apply CBR Traffic over both UDP connections
9. Choose distance vector routing protocol to transmit data from sender to receiver.
10. Schedule events and run the program

PROGRAM:

```
set ns [new Simulator] set nr [open thro.tr w]
$ns trace-all $nr
set nf [open thro.nam w]
$ns namtrace-all $nf proc finish { }
{
  global ns nr nf
  $ns flush-trace close $nf
  close $nr
  exec nam thro.nam & exit 0
}
```

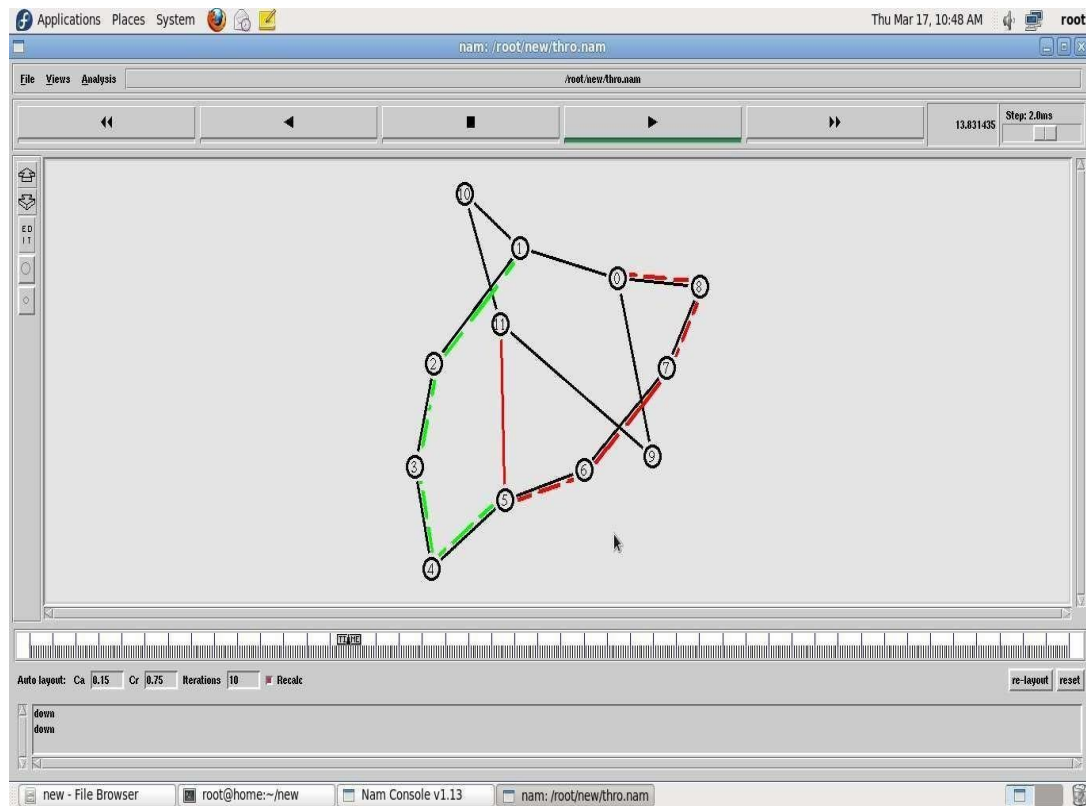
```

for { set i0 } { $i < 12 } { incr i 1 } {
set n($i) [$ns node]}
for {set i0} {$i < 8} {incr i} {
$ns duplex-link $n($i) $n([expr $i+1]) 1Mb 10ms DropTail }
$ns duplex-link $n(0) $n(8) 1Mb 10ms DropTail
$ns duplex-link $n(1) $n(10) 1Mb 10ms DropTail
$ns duplex-link $n(0) $n(9) 1Mb 10ms DropTail
$ns duplex-link $n(9) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(10) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(11) $n(5) 1Mb 10ms DropTail
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
set cbr0 [new Application/Traffic/CBR]

$scr0 set packetSize_ 500
$scr0 set interval_ 0.005
$scr0 attach-agent $udp0
set null0 [new Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp0 $null0
set udp1 [new Agent/UDP]
$ns attach-agent $n(1) $udp1
set cbr1 [new Application/Traffic/CBR]
$scr1 set packetSize_ 500
$scr1 set interval_ 0.005
$scr1 attach-agent $udp1
set null1 [new Agent/Null]
$ns attach-agent $n(5) $null1
$ns connect $udp1 $null1
$ns rtproto DV
$ns rtmodel-at 10.0 down $n(11) $n(5)
$ns rtmodel-at 15.0 down $n(7) $n(6)
$ns rtmodel-at 30.0 up $n(11) $n(5)
$ns rtmodel-at 20.0 up $n(7) $n(6)
$udp0 set fid_ 1
$udp1 set fid_ 2
$ns color 1 Red
$ns color 2 Green
$ns at 1.0 "$scr0 start"
$ns at 2.0 "$scr1 start"
$ns at 45 "finish"
$ns run

```

OUTPUT



RESULT:

Thus the Distance Vector Routing Algorithm was Simulated and studied.

EXNO: 9B**SIMULATION OF LINK STATE ROUTING****AIM:**

To simulate the link state routing algorithm using NS2

ALGORITHM:

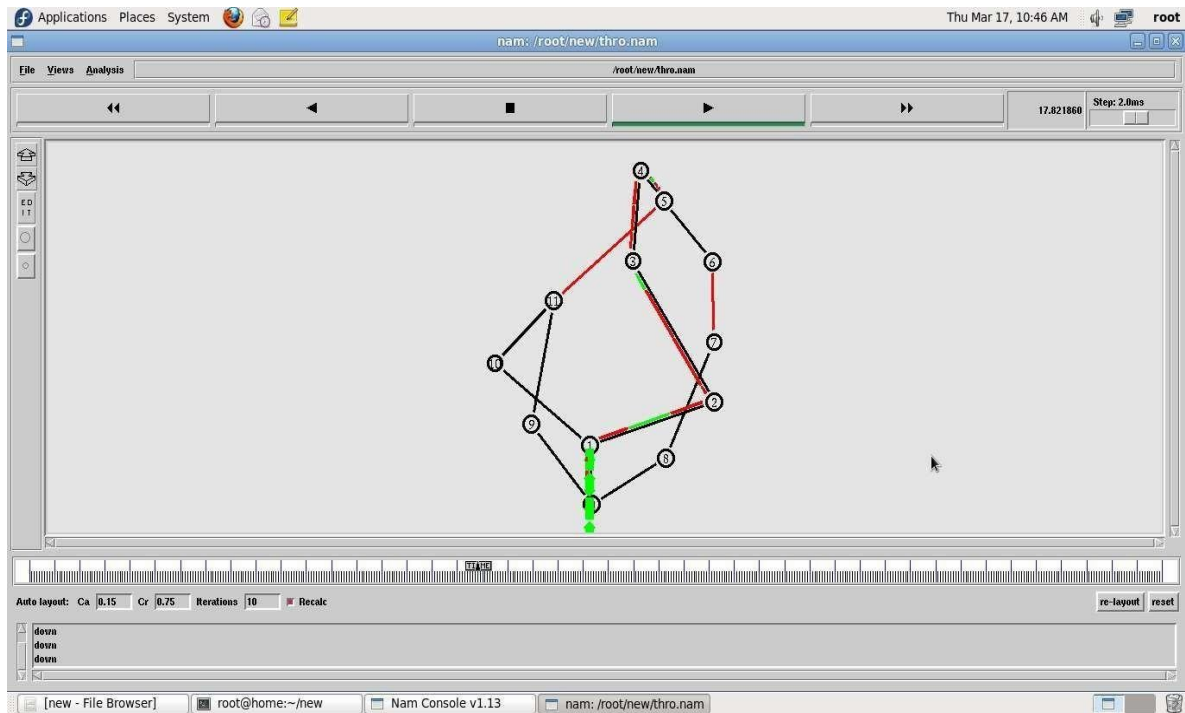
1. Create a simulator object
2. Define different colors for different data flows
3. Open a namtrace file and define finish procedure then close the trace file, and execute nam on trace file.
4. Create n number of nodes using for loop
5. Create duplex links between the nodes
6. Setup UDP Connection between n(0) and n(5)
7. Setup another UDP connection between n(1) and n(5)
8. Apply CBR Traffic over both UDP connections
9. Choose Link state routing protocol to transmit data from sender to receiver.
10. Schedule events and run the program.

PROGRAM:

```
set ns [new Simulator]
set nr [open thro.tr w]
$ns trace-all $nr
set nf [open thro.nam w]
$ns namtrace-all $nf
proc finish { } {
    global ns nr nf
    $ns flush-trace
    close $nf
    close $nr
    exec namthro.nam&
    exit 0 }
for {set i 0} {$i < 12} {incr i 1} {
    set n($i) [$ns node]}
for {set i 0} {$i < 8} {incr i} {
    $ns duplex-link $n($i) $n([expr $i+1]) 1Mb 10ms DropTail }
$ns duplex-link $n(0) $n(8) 1Mb 10ms DropTail
$ns duplex-link $n(1) $n(10) 1Mb 10ms DropTail
```

```
$ns duplex-link $n(0) $n(9) 1Mb 10ms DropTail
$ns duplex-link $n(9) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(10) $n(11) 1Mb 10ms DropTail
$ns duplex-link $n(11) $n(5) 1Mb 10ms DropTail
set udp0 [new Agent/UDP]
$ns attach-agent $n(0) $udp0
set cbr0 [new Application/Traffic/CBR]
$cbr0 set packetSize_ 500
$cbr0 set interval_ 0.005
$cbr0 attach-agent $udp0
set null0 [new Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp0 $null0
set udp1 [new Agent/UDP]
$ns attach-agent $n(1) $udp1
set cbr1 [new Application/Traffic/CBR]
$cbr1 set packetSize_ 500
$cbr1 set interval_ 0.005
$cbr1 attach-agent $udp1
set null0 [new Agent/Null]
$ns attach-agent $n(5) $null0
$ns connect $udp1 $null0
$ns rtpproto LS
$ns rtmodel-at 10.0 down $n(11) $n(5)
$ns rtmodel-at 15.0 down $n(7) $n(6)
$ns rtmodel-at 30.0 up $n(11) $n(5)
$ns rtmodel-at 20.0 up $n(7) $n(6)
$udp0 set fid_ 1
$udp1 set fid_ 2
$ns color 1 Red
$ns color 2 Green
$ns at 1.0 "$cbr0 start"
$ns at 2.0 "$cbr1 start"
$ns at 45 "finish"
$ns run
```

OUTPUT



RESULT:

Thus the Link State Routing Algorithm was Simulated and studied.

EXNO: 10 ERROR CORRECTION CODE CYCLIC REDUNDANCY CHECK

AIM:

To simulate Cyclic Redundancy check.

ALGORITHM:

1. Enter the number of data bits
2. Enter the number of generator bits+
3. Enter the data bits
4. Enter the generator bits
5. Print the code bits
6. Stop the program

PROGRAM:

```
import java.util.Scanner;
class CRC
{
public static void main(String args[])
{
Scanner sc=new Scanner(System.in);
int m,g[],n,d[],z[],r[],msb,i,j,k;
System.out.print("Enter no. of data bits : ");
n=sc.nextInt();
System.out.print("Enter no. of generator bits : ");
m=sc.nextInt();
d=new int[n+m];
g=new int[m];
System.out.print("Enter data bits : ");
for(i=0;i<n;i++)
d[i]=sc.nextInt();
System.out.print("Enter generator bits : ");
for(j=0;j<m;j++)
g[j]=sc.nextInt();
for(i=0;i<m-1;i++)
d[n+i]=0;
r=new int[m+n];
for(i=0;i<m;i++)
r[i]=d[i];
z=new int[m];
for(i=0;i<m;i++)
z[i]=0;
for(i=0;i<n;i++) { k=0;
msb=r[i];
for(j=i;j<m+i;j++)
{
if(msb==0)
r[j]=xor(r[j],z[k]);
```

```

else
r[j]=xor(r[j],g[k]);
k++;
}
r[m+i]=d[m+i];
}
System.out.print("The code bits added are : ");
for(i=n;i<n+m-1;i++)
{
d[i]=r[i];
System.out.print(d[i]);
}
System.out.print("\nThe code data is : ");
for(i=0;i<n+m-1;i++)
{
System.out.print(d[i]);
}
}
}
public static int xor(int x,int y)
{
if(x==y)
return(0);
else
return(1);
}}

```

OUTPUT:

EnTer no. of data bits : 14 Enter no. of generator bits : 4 Enter data bits : 1

1
0
1
0
0
1
1
1
0
1
1
0
0
0

Enter generator bits :

1
0
1
1

The code bits added are : 100

The code data is : 11010011101100100

Result:

Thus the java program to simulate Cyclic Redundancy check was done and output verified

EXNO: 11A

IMPLEMENTATION OF SLIDING WINDOW PROTOCOL

AIM:

To write a java program to perform Sliding Window protocol.

ALGORITHM:

- 1: Start the program
- 2: Sender keeps track of outstanding frames and updates the variable and window as acknowledgements arrive.
- 3: Receiver sliding window waits for a specific frame to arrive in a specific order and frame arriving out of order is discarded and needs to be resent.
- 4: Commence transmission.
- 5: Deal with damaged or lost frame.
- 6: Deal with damaged or lost acknowledgement.
- 7: Deal with delayed acknowledgement.
- 8: Update sender windows
- 9: Stop the program.

PROGRAM:

SENDER PROGRAM

```
import java.net.*;
import java.io.*;
import java.rmi.*;

public class slidsender
{
    public static void main(String a[])throws Exception
    {
        ServerSocket ser=new ServerSocket(10);
        Socket s=ser.accept();
        DataInputStream in=new DataInputStream(System.in);

        DataInputStream in1=new DataInputStream(s.getInputStream());

        String sbuff[]=new String[8];

        PrintStream p;
        int sptr=0,sws=8,nf,ano,i;

        String ch;
```

```

do
{
p=new PrintStream(s.getOutputStream());

System.out.print("Enter the no. of frames : ");
nf=Integer.parseInt(in.readLine());

p.println(nf); if(nf<=sws-1)
{

System.out.println("Enter "+nf+" Messages to be send\n");
for(i=1;i<=nf;i++)
{
sbuff[sptr]=in.readLi
ne();
p.println(sbuff[sptr]);
sptr=++sptr%8;
}
sws-=nf;

System.out.print("Acknowledgment received");

ano=Integer.parseInt(in1.readLine());

System.out.println(" for "+ano+" frames");

sws+=nf;

}
else
{
System.out.println("The no. of frames exceeds window size");
break;
}
System.out.print("\nDo you wants to send some more frames : ");
ch=in.readLine();
p.println(ch);
}
while(ch.equals("yes"
));
s.close();
}
}

```

RECEIVER PROGRAM

```
import java.net.*;
import java.io.*;
class slidreceiver
{
    public static void main(String a[])throws Exception
    {
        Socket s=new Socket(InetAddress.getLocalHost(),10);
        DataInputStream in=new DataInputStream(s.getInputStream());
        PrintStream p=new PrintStream(s.getOutputStream());
        int i=0,rptr=-1,nf,rws=8;

        String rbuf[]=new String[8];
        String ch;
        System.out.println();
        do
        {
            nf=Integer.parseInt(in.readLine());
            if(nf<=rws-1)
            {
                for(i=1;i<=nf;i++)
                {
                    rptr=++rptr%8;
                    rbuf[rptr]=in.readLine();
                    System.out.println("The received Frame " +rptr+" is : "+rbuf[rptr]);
                }
                rws-=nf;
                System.out.println("\nAcknowledgment sent\n");
                p.println(rpctr+1); rws+=nf; }
            else
                break;
            ch=in.readLine();
        }
        while(ch.equals("yes"));
    }
}
```

OUTPUT:**//SENDER OUTPUT**

Enter the no. of frames : 4

Enter 4 Messages to be send

hiii

how r u

i am fine

how is everyone

Acknowledgment received for 4 frames

Do you wants to send some more frames : no

//RECEIVER OUTPUT

The received Frame 0 is : hiii

The received Frame 1 is : how r u

The received Frame 2 is : iam fine

The received Frame 3 is : how is everyone

RESULT:

Thus the program for implementation of SLIDING WINDOW PROTOCOL was executed and the output is verified successful y.

AIM:

To write a java program to perform Stop and Wait protocol.

ALGORITHM:

- 1: Start the program
- 2: Sender keeps track of outstanding frames and updates the variable and window as acknowledgements arrive.
- 3: Receiver sliding window waits for a specific frame to arrive in a specific order and frame arriving out of order is discarded and needs to be resent.
- 4: Commence transmission.
- 5: Deal with damaged or lost frame.
- 6: Deal with damaged or lost acknowledgement.
- 7: Deal with delayed acknowledgement.
- 8: Update sender windows
- 9: Stop the program.

PROGRAM**//SENDER PROGRAM**

```
import java.io.*;
import java.net.*;
public class Sender{
    Socket sender;
    ObjectOutputStream out;
    ObjectInputStream in;
    String packet,ack,str, msg;
    int n,i=0,sequence=0;
    Sender(){ }
    public void run(){
        try{
            BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
            System.out.println("Waiting for Connection... ");
            sender = new Socket("localhost",2004);
            sequence=0;
            out=new ObjectOutputStream(sender.getOutputStream());
            out.flush();
            in=new ObjectInputStream(sender.getInputStream());
```

```

str=(String)in.readObject();
System.out.println("reciver > "+str);
System.out.println("Enter the data to send. .. ");
packet=br.readLine();
n=packet.length();
do{
try{
if(i<n){
msg=String.valueOf(sequence);
msg=msg.concat(packet.substring(i,i+1));
}
else if(i==n){
msg="end";out.writeObject(msg);break;
}
out.writeObject(msg);
sequence=(sequence==0)?1:0;
out.flush();
System.out.println("data sent>"+msg);
ack=(String)in.readObject();
System.out.println("waiting for ack    \n\n");
if(ack.equals(String.valueOf(sequence))) {
i++;
System.out.println("receiver > "+" packet recieved\n\n");
}
else {
System.out.println("Time out resending data    \n\n");
sequence=(sequence==0)?1:0;
}
} catch (Exception e) {}
}while(i<n+1);
System.out.println("All data sent. exiting.");
} catch (Exception e) {}
finally {
try {
in.close();
out.close();
sender.close();
}
catch (Exception e) {}
}
}
public static void main(String args[]) {
Sender s=new Sender();
s.run();
}
}

```


//RECEIVER PROGRAM

```
import java.io.*;
import java.net.*;
public class Reciever{
    ServerSocket reciever;
    Socket connection=null;
    ObjectOutputStream out;
    ObjectInputStream in;
    String packet,ack,data="";
    int i=0,sequence=0;
    Reciever() {}
    public void run(){
        try{
            BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
            reciever = new ServerSocket(2004,10);
            System.out.println("waiting for connection...");
            connection=reciever.accept();
            sequence=0;
            System.out.println("Connection established :");
            out=new ObjectOutputStream(connection.getOutputStream());
            out.flush();
            in=new ObjectInputStream(connection.getInputStream());
            out.writeObject("connected .");
            Do
            {
                try
                {
                    packet=(String)in.readObject();
                    if(Integer.valueOf(packet.substring(0,1))==sequence)
                    {
                        data+=packet.substring(1);
                        sequence=(sequence==0)?1:0;
                        System.out.println("\n\nreceiver >"+packet);
                    }
                    else
                    {
                        System.out.println("\n\nreceiver >"+packet +" duplicate data");
                    }
                }
                if(i<3){
                    out.writeObject(String.valueOf(sequence));i++;
                }
                else{ out.writeObject(String.valueOf((sequence+1%2)));
                    i=0;
                }
            }
            catch(Exception e) {}
        }while(!packet.equals("end"));
        System.out.println("Data recived="+data);
        out.writeObject("connection ended .");
    }
    catch(Exception e) {}
}
```

```

finally{
try{
in.close();
out.close();
reciever.close();
}
catch(Exception e){}
}
}
public static void main(String args[]){
Reciever s=new Reciever();
while(true){
s.run();
}
}
}

```

OUTPUT:

//SENDER OUTPUT

```

Waiting for Connection....
reciver > connected .
Enter the data to send....
myname
data sent>0m
waiting for ack.....
receiver > packet recieved
data sent>1y
waiting for ack.....
receiver > packet recieved
data sent>0n
waiting for ack.....
receiver > packet recieved
data sent>1a
waiting for ack.....
Time out resending data....
data sent>1a
waiting for ack.....
receiver > packet recieved
data sent>0m
waiting for ack.....
receiver > packet recieved
data sent>1e
waiting for ack.....
receiver > packet recieved
All data sent. exiting.

```

//RECEIVER OUTPUT

```
waiting for connection...
Connection established :
receiver >0m
receiver >1y
receiver >0n
receiver >1a
receiver >1a duplicate data
receiver >0m
receiver >1e
Data recived=myname
waiting for connection...
```

RESULT:

Thus the program for implementation of STOP AND WAIT PROTOCOL was executed and the output is verified successfully.

AIM

To simulate a sliding window protocol that uses Go Back N ARQ.

ALGORITHM:**SENDER**

1. Create a server socket
2. Assume the sending window size as 7 ($m=3$).
3. If receiver is ready, initialize sender's frame sequence to 0.
4. Get data from user.
5. Send it to the receiver along with sequence number.
6. Increment sequence number by 1.
7. Repeat steps 4–6 until all frames have been sent.
8. Wait for acknowledgements.
9. If all acknowledgements have arrived then go to step 12.
10. Set sequence number to earliest outstanding frame for which there is no ACK.
11. Go to step 4.
12. Stop

RECEIVER

1. Indicate to sender, the readiness to accept frames.
2. Initialize receiver's expected frame sequence to 0.
3. Accept the incoming frame.
4. If frame's sequence receiver's sequence then go to step 7.
5. Send an acknowledgement.
6. Repeat steps 3–6 until all frames are received in sequence and go to step 9
7. Discard frame, thereby force the sender to retransmit.
8. Go to step 3.
9. Stop

PROGRAM

Go Back N Sender

```
import java.io.*; import java.net.*;
public class GBNSend
{
String output[] = new String[7];
void sendMsg(BufferedReader buff, BufferedReader in, PrintWriter out, int x) throws Exception
{
try
{
for(int i=x; i<7; i++)
{
System.out.print("Content for frame" + i + " : ");
output[i] = in.readLine(); out.println(output[i]);
out.flush();
}
}
catch(Exception e)
{
System.out.println("exception is" + e);
}
}
public static void main(String[] args) throws Exception
{
GBNSend g = new GBNSend();
ServerSocket ssoc = new ServerSocket(6000);
Socket soc = ssoc.accept();
String input[] = new String[7];
BufferedReader buff = new BufferedReader(new InputStreamReader(soc.getInputStream()));
PrintWriter out = new PrintWriter(new OutputStreamWriter(soc.getOutputStream()));
BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
System.out.println("Window size 7");
System.out.println("Enter message in format-- SeqNoData");
int x = 0;
g.sendMsg(buff,in,out,0);
for(int j=0; j<7; j++)
{
String s = new String();
input[j] = buff.readLine(); s = j + "";
if(input[j].startsWith(s))
System.out.println("Ack for frame" + j);
else
{
System.out.println("Frame" + j+ " lost/corrupt. Go Back & resend");
x = j;
g.sendMsg(buff,in,out,j);
--j;
}
} soc.close(); ssoc.close();
}
```

//Go Back N Receiver--GBNReceive.java

```
import java.io.*;
import java.net.*;
public class GBNReceive
{
    String input[] = new String[7];
    void ReceiveMsg(BufferedReader buff, PrintWriter out, int x) throws Exception
    {
        try
        {
            for(int i=x; i<7; i++)
                input[i] = buff.readLine();
        }
        catch(Exception e)
        {
            System.out.println("exception is" + e);
        }
    }
    public static void main(String[] args) throws Exception
    {
        GBNReceive g = new GBNReceive();
        Socket soc = new Socket("localhost", 6000);
        BufferedReader buff = new BufferedReader(new InputStreamReader(soc.getInputStream()));
        PrintWriter out = new PrintWriter(new OutputStreamWriter(soc.getOutputStream()));
        int x = 0;
        g.ReceiveMsg(buff,out,0);
        for(int j=x; j<7; j++)
        {
            String s = new String(); s = j + "";
            if(g.input[j].startsWith(s))
            {
                String rmsg = g.input[j];
                rmsg = rmsg.substring(1,rmsg.length());

                System.out.println("Frame" + j+ "data is " + rmsg);

                out.println(j + "ack");
                out.flush();
            }
            else
            {
                out.println("no ack");
                out.flush(); x = j; g.ReceiveMsg(buff,out,j);
                --j;
            }
        }
        soc.close();
    }
}
```

OUTPUT:

Sender

```
$ javac GBNSend.java
$ java GBNSend
Window size 7
Enter message in format--SeqNoData
Content for frame0 : 0hi
Content for frame1 : 1hello
Content for frame2 : 2i am here
Content for frame3 : 3where r u?
Content for frame4 : here only
Content for frame5 : 5can u see me
Content for frame6 : 6which direction
Ack for frame0
Ack for frame1
Ack for frame2
Ack for frame3
Frame4 lost/corrupt. Go Back & resend
Content for frame4 : 4here only
Content for frame5 : 5which direction
Content for frame6 : ok
Ack for frame4
Ack for frame5
Frame6 lost/corrupt. Go Back & resend
Content for frame6 : 6South
Ack for frame6
```

Receiver

```
$ javac GBNReceive.java
$ java GBNReceive
Frame0 data is hi
Frame1 data is hello
Frame2 data is iamhere
Frame3 data is where r u?
Frame4 data is here only
Frame5 data is which direction
Frame6 data is South
```

RESULT:

Thus the java program to simulate Go-Back-N ARQ was done and output verified

